International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Case Studies: Investigating Memory Evidence

Student Name: Kafayat Omolara Animashawun
Student ID: 2025/FWSD/11317
Instructor Name: Aminu Idris
Date of Submission: 19/10/2026

EXECUTIVE SUMMARY

This investigation examined a Windows memory image named memdumpWin7.mem to determine whether the available memory artefacts supported a suspected transfer of a confidential document to removable storage. The examination was conducted offline using Volatility 2.6.1 in a Kali Linux environment. The investigation focused on process activity, command history, network artefacts, Windows registry records, removable-media identification, drive-letter mappings, and user-profile information.

The recovered console history contained the command copy secret_file.docx F: and the message 1 file(s) copied. The console context showed that the command was entered after changing the working directory to C:\Users\IEUser\Downloads. Registry artefacts identified a device described as General UDisk USB Device and associated the F: drive mapping with that USB storage device. These findings provide mutually supporting evidence consistent with a file-copy operation targeting removable storage.

Network analysis identified an OpenSSH service process, sshd.exe (PID 2016), listening on TCP port 22. Two closed TCP records connected the local address 192.168.56.8 to 192.168.56.5 on port 80, but these records do not establish that the document was transmitted over a network. The available evidence also does not independently establish the contents or sensitivity of secret_file.docx, whether the destination file remained intact,

whether the transfer was authorised, or the identity of the individual who entered the command.

The original memory image and working copy were verified to have identical SHA-256 hashes and file sizes. The selected Volatility profile was Win7SP1x86, consistent with Windows 7 SP1 rather than the Windows 10 operating system described in the scenario. ShellBags analysis could not be completed because the required Python cryptography module was unavailable. These limitations are documented in the report.

Overall, the evidence supports a conclusion that the captured console state records a file-copy operation to a drive mapped to a USB storage device. The conclusion that an unauthorised transfer of confidential information occurred remains unproven.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment was to conduct a structured memory-forensics investigation of the supplied Windows memory image to determine whether recoverable digital artefacts could clarify a suspected file-transfer incident. The examination aimed to reconstruct relevant workstation activity by correlating process information, command history, network records, registry artefacts, removable-media records, and user-profile evidence.

The investigation also aimed to demonstrate sound forensic practice by preserving the supplied evidence, verifying the integrity of the working copy, recording the commands and outputs used, maintaining a chronological account of relevant artefacts, and distinguishing observed facts from investigative interpretations. Particular care was taken not to treat a recovered command, registry entry, or process record as conclusive proof of unauthorized activity or individual responsibility.

OBJECTIVES OF THE ASSIGNMENT

The specific objectives of the investigation were to:

1. Preserve the supplied memory image and verify the integrity of the working copy using file-size and SHA-256 comparisons.

2. Identify and justify the appropriate Volatility memory profile using the actual image output.
3. Examine active and residual process information to identify processes potentially relevant to the suspected incident.
4. Analyse available network artefacts, including local and foreign endpoints, connection states, process associations, and listening services.
5. Recover command-line and console history to identify commands, directory changes, drive references, and recorded command results relevant to the suspected transfer.
6. Examine USBSTOR, MountedDevices, and MountPoints2 registry artefacts to identify removable storage and correlate it with the F: drive mapping and user-profile records.
7. Review available account-profile and File Explorer-related artefacts, documenting any limitations caused by unavailable plugins or missing records.
8. Construct a UTC timeline of relevant events and artefact timestamps, distinguishing recorded timestamps from inferred user actions.
9. Assess whether the evidence supports a suspected transfer, identify alternative explanations, and state a conclusion with an appropriate confidence level.
10. Maintain an evidence log linking findings to the relevant Volatility commands, output files, registry paths, process records, and screenshots.

METHODOLOGY OF THE ASSIGNMENT

The investigation followed a structured, offline memory-forensics methodology intended to preserve the original evidence and make the analysis reproducible.

1. Evidence preparation and integrity verification: The supplied memdumpWin7.mem file was maintained in an original evidence directory, with a separate working copy created for analysis. The SHA-256 hashes and file sizes of both files were calculated and compared. Both files produced the hash 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e and had a size of 1,073,676,288 bytes. The matching values established that the working copy was byte-for-byte identical to the original at the time of verification.

2. Memory profile identification: Volatility's imageinfo plugin was used to examine the memory image and identify suitable candidate profiles. The Win7SP1x86 profile was selected for subsequent analysis based on the available image information and usable output from relevant plugins. The results were consistent with Windows 7 SP1, which differs from the Windows 10 description in the case scenario. The discrepancy was recorded rather than assuming the scenario description accurately represented the supplied image.
3. Process and network analysis: The pslist, pstree, and cmdline plugins were used to examine processes, process relationships, and command-line arguments. The psscan output was also reviewed, but dependency-related plugin limitations meant that it did not provide usable process-scan results. The netscan plugin was used to examine recovered network artefacts, including listening ports, endpoint addresses, connection states, and process associations. No endpoint was contacted during the investigation.
4. Command-history analysis: The cmdscan and consoles plugins were used to examine recovered command-interpreter history and console buffers. The results were reviewed for directory changes, file-copy commands, drive references, and console messages. The recovered command copy secret_file.docx F: and its associated 1 file(s) copied. message were assessed alongside the USB registry evidence.
5. Registry and removable-media analysis: The hivelist plugin was used to locate registry hives in memory. The printkey plugin was then used to examine USBSTOR, MountedDevices, MountPoints2, ProfileList, and relevant user-profile records. These results were correlated to assess the relationship between the General UDisk USB Device, the F: drive mapping, the associated volume GUID, and the IEUser profile. Registry timestamps were treated as artefact timestamps rather than definitive proof of device insertion or file-copy execution.
6. User-profile and File Explorer review: ProfileList and user-hive artefacts were examined to identify the profiles associated with the recovered SIDs and to review available RecentDocs, RunMRU, TypedPaths, and MountPoints2 records. ShellBags analysis was attempted but could not be completed because Volatility

could not import the required Crypto.Hash module in the Python 2 environment. No ShellBags findings were inferred from the failed attempt.

7. Timeline reconstruction and reporting: Relevant process creation times, registry key update times, and network artefact timestamps were organised chronologically in UTC. The timeline distinguished directly observed records from interpretations and documented where an exact event time could not be established. Findings were evaluated collectively, with alternative explanations and evidential limitations recorded before forming the final conclusion.

TOOLS AND RESOURCES USED

The following tools and resources were used during the investigation:

Tool or resource	**Purpose in the investigation**
Kali Linux	Operating environment for the offline forensic examination.
Python 2.7	Runtime used to execute Volatility 2.
Volatility 2.6.1	Memory-forensics framework used to analyse the supplied Windows memory image.
imageinfo	Identified candidate memory profiles and provided image-related information.
pslist	Examined recoverable active process records.
psscan	Attempted to identify process records through memory scanning; output was limited by plugin dependencies.
pstree	Examined parent-child process relationships.
cmdline	Retrieved command-line information associated with relevant processes.
cmdscan	Examined recoverable command-interpreter history.
consoles	Examined console objects, command history, and recovered screen-buffer text.
netscan	Examined network artefacts, including endpoints, connection states, and listening services.

hivelist	Located registry hives available in the memory image.
printkey	Examined USBSTOR, MountedDevices, MountPoints2, ProfileList, and related registry records.
sha256sum	Calculated SHA-256 hashes for the original memory image and working copy.
stat	Verified the file sizes of the original memory image and working copy.
ShellBags plugin availability check	Tested whether the ShellBags plugin could be loaded; analysis was blocked by the missing Crypto.Hash dependency.
Supplied memdumpWin7.mem	Primary memory evidence examined during the investigation.
Saved Volatility output files	Preserved the results used to support and reproduce the reported findings.

Investigation Workspace Preparation

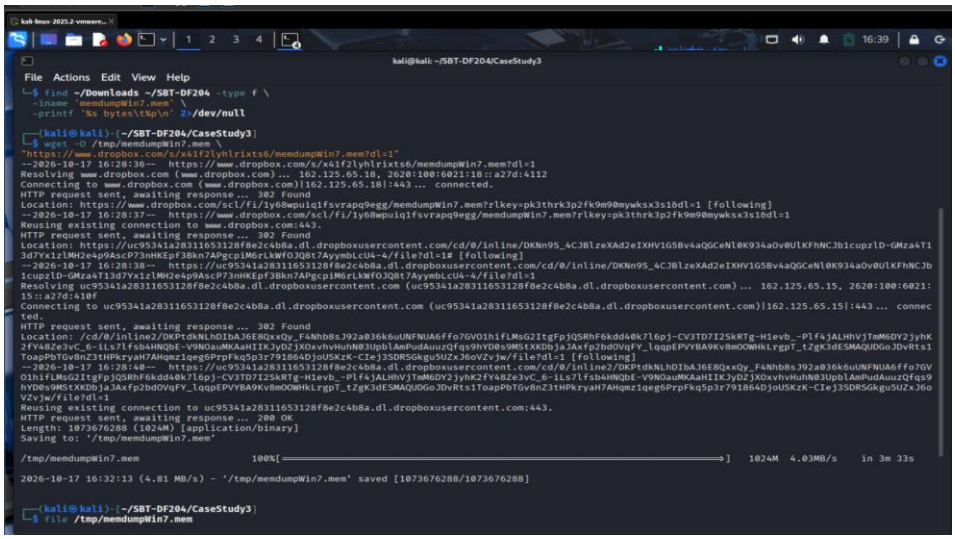
A structured investigation workspace was created in the Kali Linux virtual machine running under VMware Workstation. The workspace was located at /home/kali/SBT-DF204/CaseStudy3 and organised into directories for original evidence, working evidence, investigation outputs, notes, timeline records, screenshots, and the final report. The directory structure was created using the Linux mkdir -p command and verified using find . -type d | sort.



This organization provided a consistent location for preserving the original memory image separately from the working copy and for storing investigation results and report materials. The screenshot documents the preparation of the workspace before or during the forensic examination; it does not independently establish evidence integrity or demonstrate that the listed directories contain the expected files.

Acquisition of the Supplied Memory Image

The supplied memory image, memdumpWin7.mem, was downloaded from the course-provided Dropbox link using the wget utility in the Kali Linux virtual machine. The terminal output records the download beginning on 17 October 2026 at 16:28:36 and completing at 16:32:13. The downloaded file was saved temporarily to /tmp/memdumpWin7.mem. The download utility reported a completed transfer of 1,073,676,288 bytes, equivalent to 1,024 MiB (1 GiB).

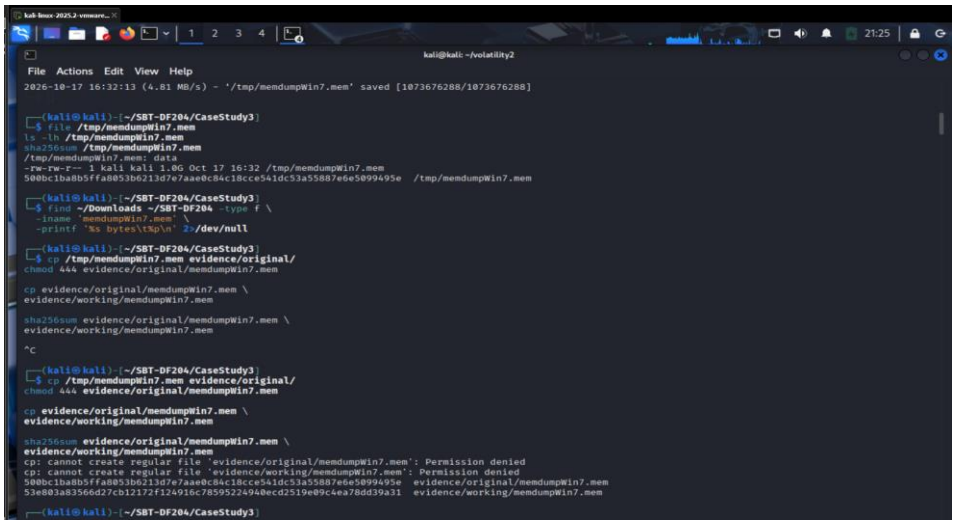


Following acquisition, the memory image was organised into the investigation workspace, with separate locations for the original evidence and working copy. Subsequent SHA-256 verification produced the same hash for both files: 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e. Their matching sizes and hashes demonstrate that the original and working files were byte-for-

byte identical at the time of verification. The download record and subsequent integrity check provide supporting documentation for the evidence-acquisition process. The recorded download completion time is an acquisition timestamp and should not be confused with the time of the suspected file-transfer activity represented in the memory image.

Evidence Acquisition and Integrity Verification

The memory image, memdumpWin7.mem, was downloaded from the course-provided resource to the temporary directory /tmp/ within the Kali Linux forensic environment. The terminal output recorded the completion of the download on 17 October 2026 at 16:32:13. The downloaded file had a reported size of 1,073,676,288 bytes, equivalent to approximately 1 GiB. The Linux file utility identified the file type as generic binary data, which is consistent with the file being a raw memory image but does not, by itself, establish its forensic validity.



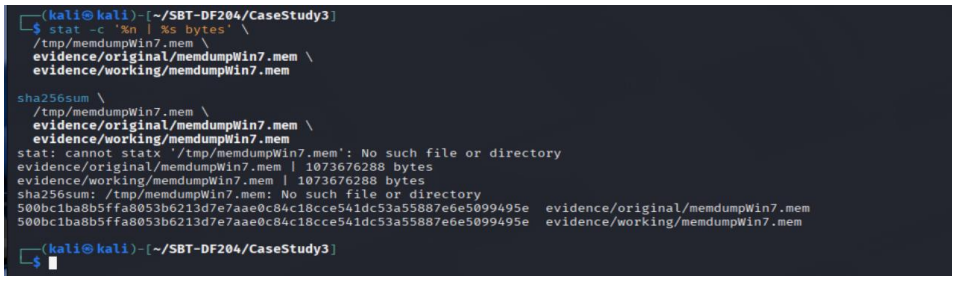
Following acquisition, the SHA-256 checksum of the downloaded file was calculated as 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e. The file was then copied into the case's evidence/original/ directory, with an attempt to set read-only permissions using chmod 444. A separate copy was intended for the evidence/working/ directory to support analysis while preserving the original evidence.

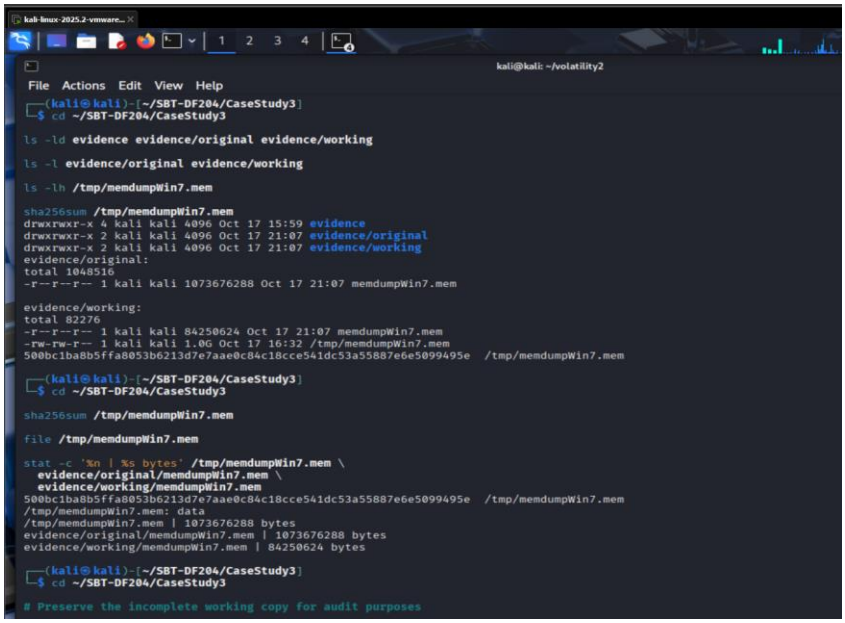
However, the terminal output records Permission denied errors during subsequent copy attempts. The displayed checksums also differ: the original evidence path shows 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e, while the working evidence path shows 53e803a83566d27cb12172f124916c78595224940ecd2519e09c4ea78dd39a31. Therefore, this screenshot does not establish that the two files were identical at the time of verification. The discrepancy must be resolved by checking the files' ownership, permissions, sizes, and checksums before the evidence copies are treated as verified.

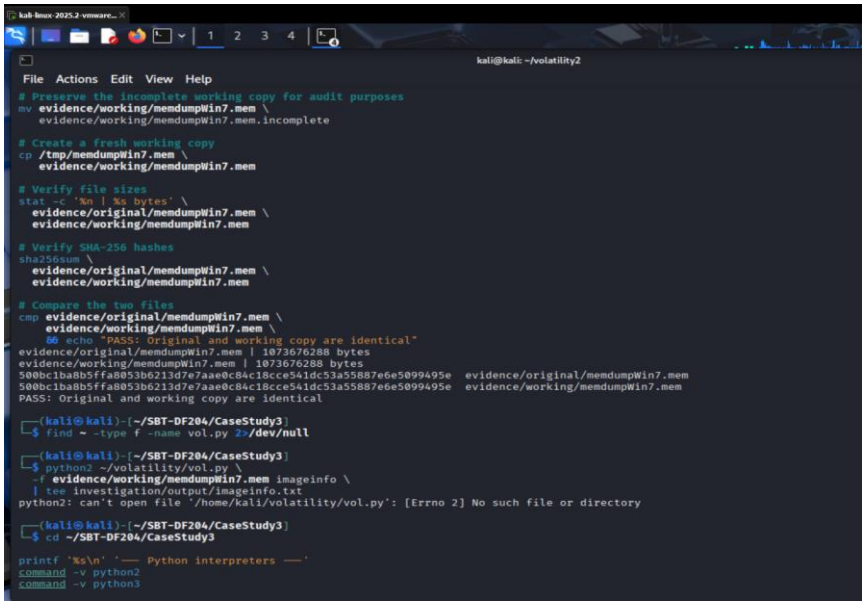
The initial checksum of the downloaded file provides a reference value against which the stored evidence can be checked. The integrity record should document the final verified checksums, file sizes, and any corrective actions taken. No claim of matching original and working copies should be made until the checksums have been confirmed to match.

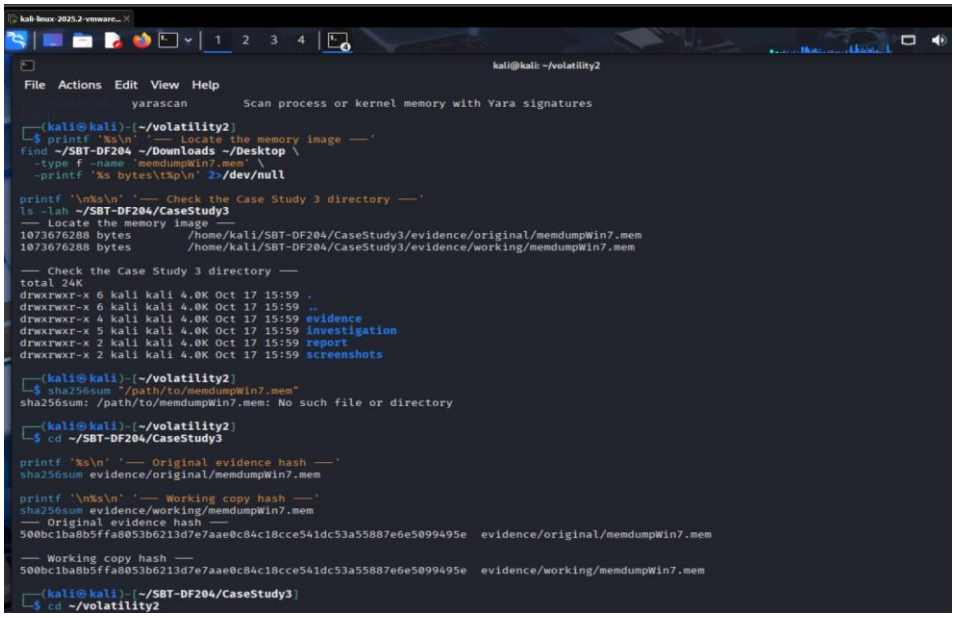
Evidence Integrity Verification and Corrective Action

Following the initial acquisition, the memory image was stored in the case's evidence/original/ directory, and a separate working copy was created in evidence/working/. During verification, an incomplete working copy was identified: its size was 84,250,624 bytes, whereas the original image was 1,073,676,288 bytes. The initial copy operation had encountered permission-related errors, and the incomplete copy was therefore not accepted as a verified forensic working copy. To correct the discrepancy, the working copy was recreated from the downloaded memory image. Subsequent verification confirmed that both evidence/original/memdumpWin7.mem and evidence/working/memdumpWin7.mem were 1,073,676,288 bytes in size and shared the same SHA-256 checksum: `500bc1ba8b5ffa8




kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/volatility2
File Actions Edit View Help
┌──(kali㉿kali)-[~/SBT-DF204/CaseStudy3]
└─$ cd ~/SBT-DF204/CaseStudy3
ls -ld evidence evidence/original evidence/working
drwxrwxr-x 4 kali kali 4096 Oct 17 15:59 evidence
drwxrwxr-x 2 kali kali 4096 Oct 17 21:07 evidence/original
drwxrwxr-x 2 kali kali 4096 Oct 17 21:07 evidence/working
ls -l evidence/original evidence/working
evidence/original:
total 1048516
-r--r--r-- 1 kali kali 1073676288 Oct 17 21:07 memdumpWin7.mem
evidence/working:
total 82276
-r--r--r-- 1 kali kali 84250624 Oct 17 21:07 memdumpWin7.mem
-rw-rw-r-- 1 kali kali 1.0G Oct 17 16:32 /tmp/memdumpWin7.mem
500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e /tmp/memdumpWin7.mem
ls -lh /tmp/memdumpWin7.mem
sha256sum /tmp/memdumpWin7.mem
file /tmp/memdumpWin7.mem
stat -c '%n | %s bytes' /tmp/memdumpWin7.mem \
evidence/original/memdumpWin7.mem \
evidence/working/memdumpWin7.mem
500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e /tmp/memdumpWin7.mem
/tmp/memdumpWin7.mem: data
/tmp/memdumpWin7.mem | 1073676288 bytes
evidence/original/memdumpWin7.mem | 1073676288 bytes
evidence/working/memdumpWin7.mem | 84250624 bytes
Preserve the incomplete working copy for audit purposes


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/volatility2
File Actions Edit View Help
Preserve the incomplete working copy for audit purposes
mv evidence/working/memdumpWin7.mem \
evidence/working/memdumpWin7.mem.incomplete
Create a fresh working copy
cp /tmp/memdumpWin7.mem \
evidence/working/memdumpWin7.mem
Verify file sizes
stat -c '%n | %s bytes' \
evidence/original/memdumpWin7.mem \
evidence/working/memdumpWin7.mem
Verify SHA-256 hashes
sha256sum \
evidence/original/memdumpWin7.mem \
evidence/working/memdumpWin7.mem
Compare the two files
cmp evidence/original/memdumpWin7.mem \
evidence/working/memdumpWin7.mem \
&& echo "PASS: Original and working copy are identical"
evidence/original/memdumpWin7.mem | 1073676288 bytes
evidence/working/memdumpWin7.mem | 1073676288 bytes
500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e evidence/original/memdumpWin7.mem
500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e evidence/working/memdumpWin7.mem
PASS: Original and working copy are identical
find ~ -type f -name vol.py 2>/dev/null
python2 ~/volatility/vol.py \
-f evidence/working/memdumpWin7.mem imageinfo \
| tee investigation/output/imageinfo.txt
python2: can't open file '/home/kali/volatility/vol.py': [Errno 2] No such file or directory
cd ~/SBT-DF204/CaseStudy3
printf '%s\n' '── Python interpreters ──'
command -v python2
command -v python3

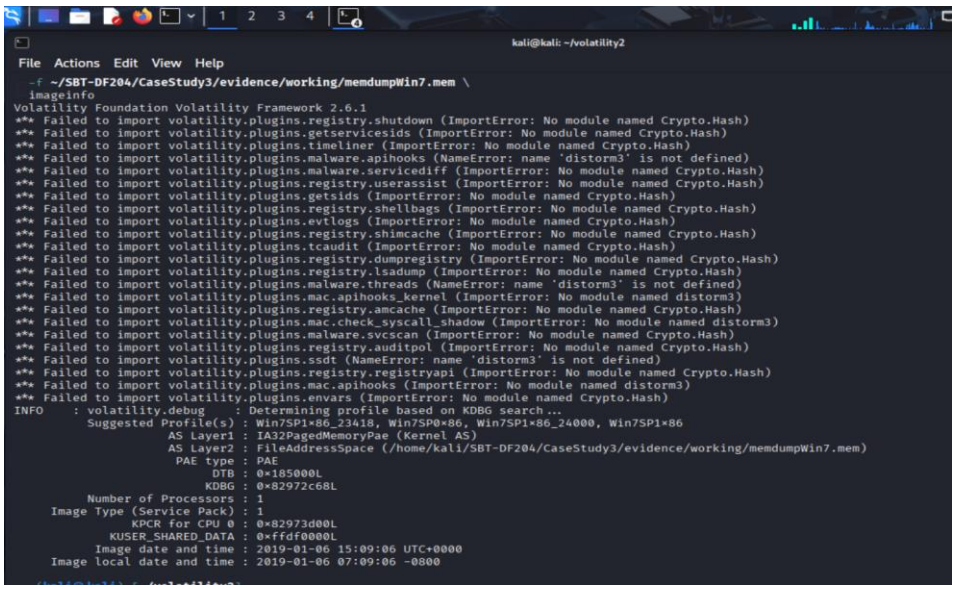


Memory Image Identification and Profile Selection

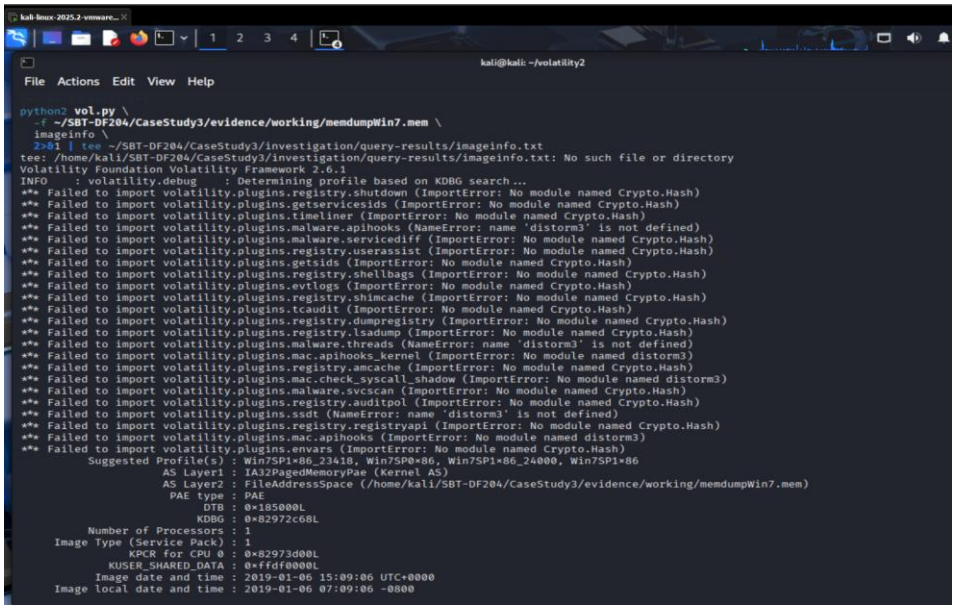
The Volatility imageinfo plugin was used to examine the supplied memory image and identify the most appropriate operating system profile for subsequent analysis. The image was processed from the working evidence path, evidence/working/memdumpWin7.mem, within the Kali Linux forensic environment.

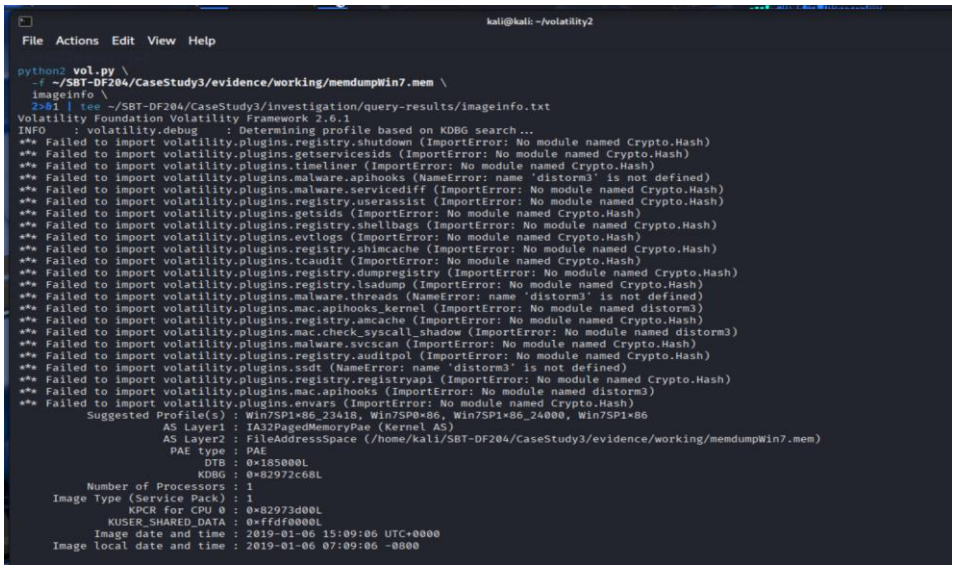
The output suggested several candidate profiles: Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, and Win7SP1x86. These results are consistent with a 32-bit Windows memory image, with Windows 7 Service Pack 1 profiles among the candidates. The profile selection was therefore based on the imageinfo results and the need to use a compatible profile for the subsequent Volatility plugins.

The image timestamp reported by Volatility was 6 January 2019 at 15:09:06 UTC, corresponding to 6 January 2019 at 07:09:06 with a local offset of UTC−08:00. The reported local timestamp is eight hours behind UTC. For consistency, the investigation timeline uses UTC timestamps. This timestamp represents the time metadata identified in the memory image; it should not automatically be interpreted as the exact time of every activity under investigation.



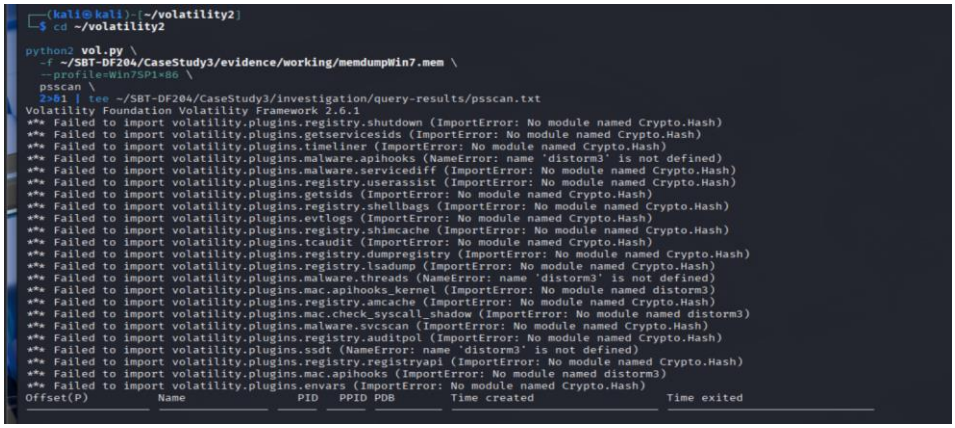
The analysis proceeded using the Win7SP1x86 profile. This selection is consistent with the suggested profiles, although imageinfo generates candidates rather than conclusively proving the correct profile. The selected profile was used for subsequent process, command history, network, and registry examinations. The supplied assignment scenario describes a Windows 10 system, but the memory-image metadata instead indicates a Windows 7-era environment. This discrepancy was recorded as a limitation, and the investigation relied on the evidence image rather than assuming the scenario description was technically accurate.





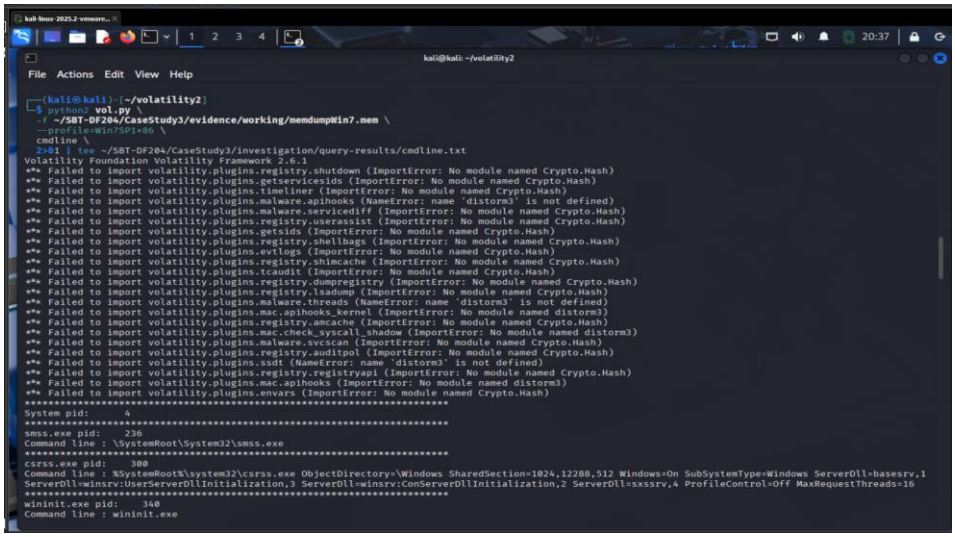
Process Enumeration and Command-Line Analysis

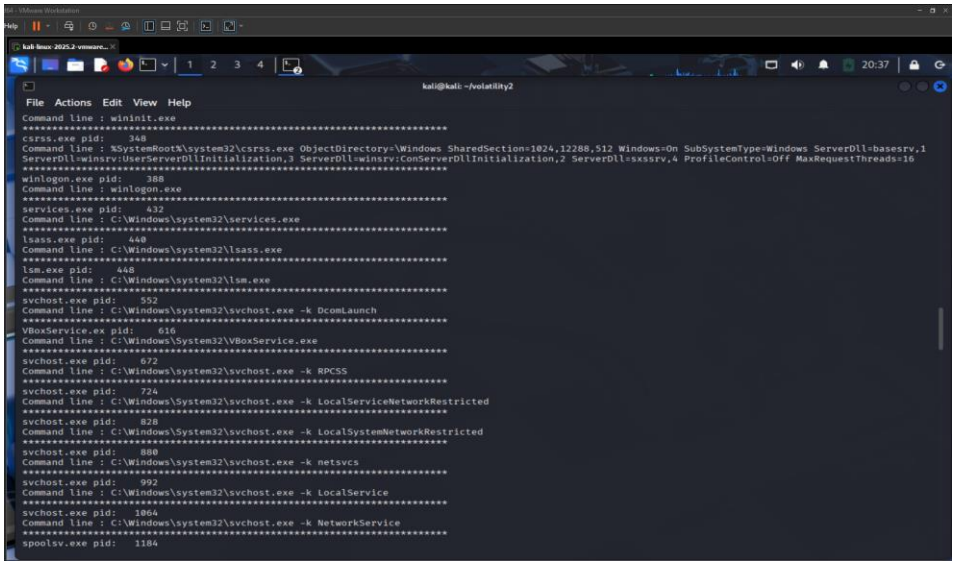
The process enumeration and command-line analysis were conducted using Volatility Framework 2.6.1 against the working copy of the Windows 7 memory image, memdumpWin7.mem, using the Win7SP1x86 profile. The psscan plugin was executed to identify process objects within the memory image, including potential processes that might not appear in the active process list. However, the captured output did not return any process entries, displaying only warnings relating to missing Python dependencies, including Crypto.Hash and distorm3. Consequently, the psscan result was inconclusive and could not independently establish whether hidden or terminated processes were present.

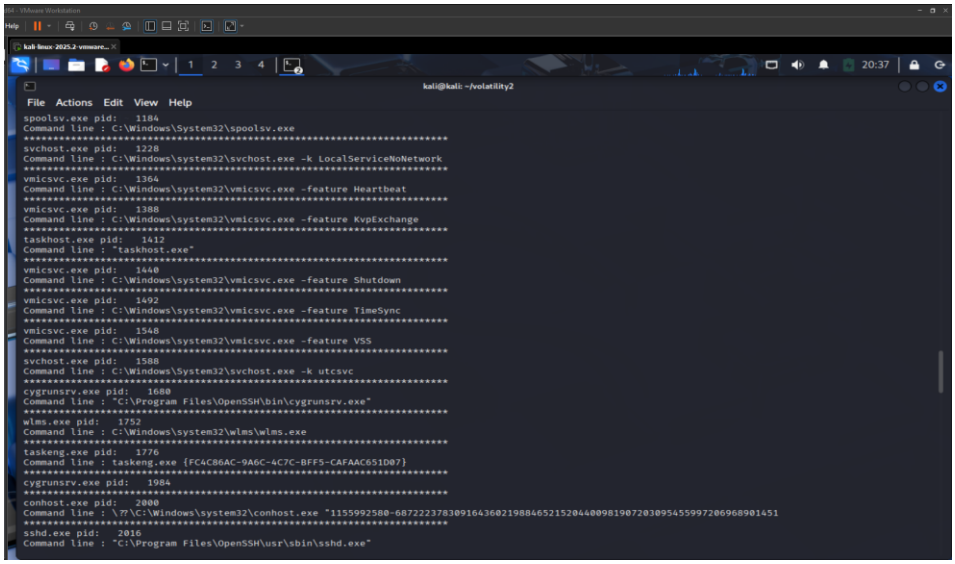


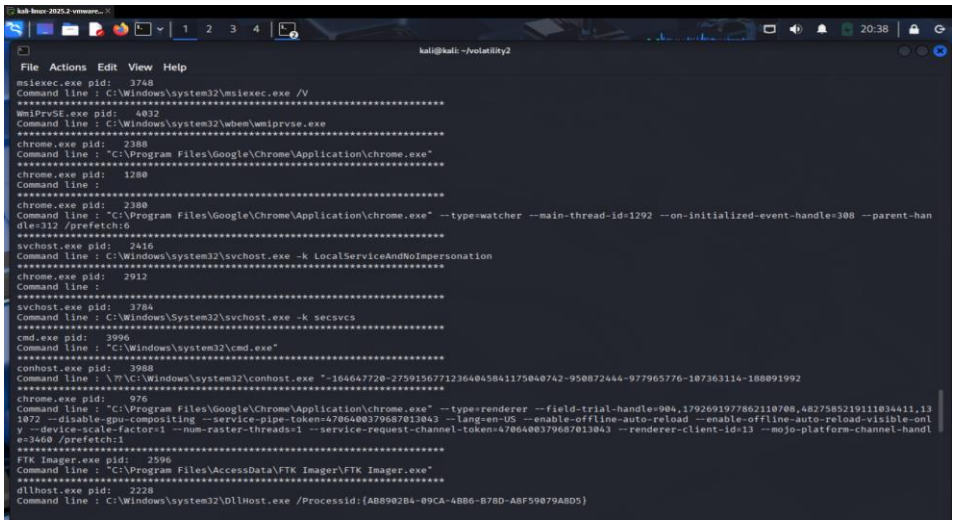
The investigation then proceeded with the cmdline plugin, which successfully recovered command-line information for multiple processes. These included standard Windows system processes such as smss.exe, csrss.exe, wininit.exe, services.exe, and lsass.exe, as well as application and service processes associated with Google Chrome, VirtualBox, and OpenSSH. Of particular interest were cmd.exe (PID 3996), whose parent process was identified in the subsequent process-tree output as explorer.exe (PID 2524), and FTK Imager.exe (PID 2596), which was also associated with explorer.exe. The output additionally identified cygrunsrv.exe processes (PIDs 1680 and 1984) and sshd.exe (PID 2016), indicating the presence of OpenSSH-related components in the captured environment. These findings provide useful investigative leads, but the process names

and command-line arguments alone do not establish that the activity was malicious or unauthorised.



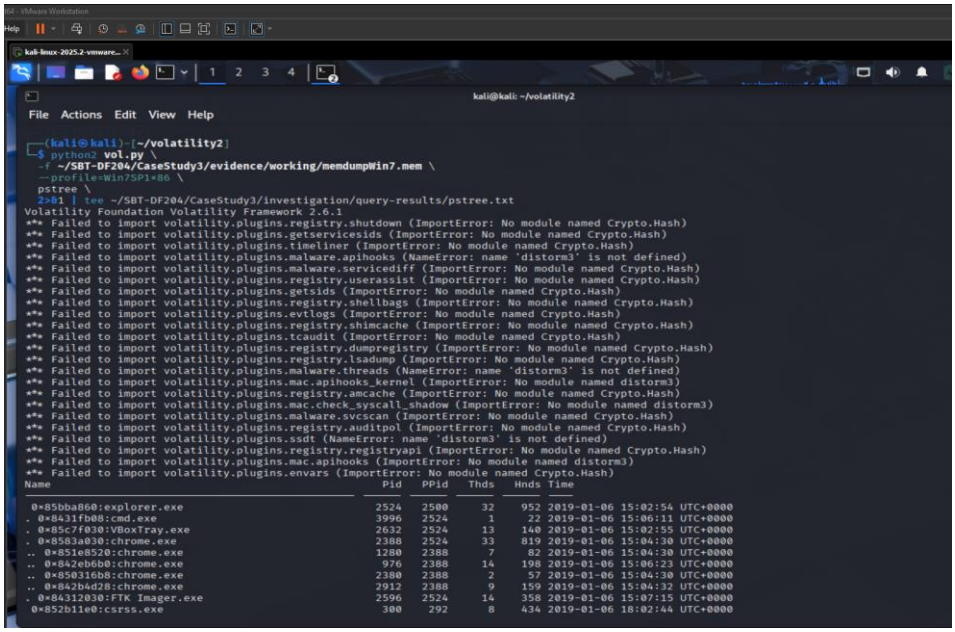


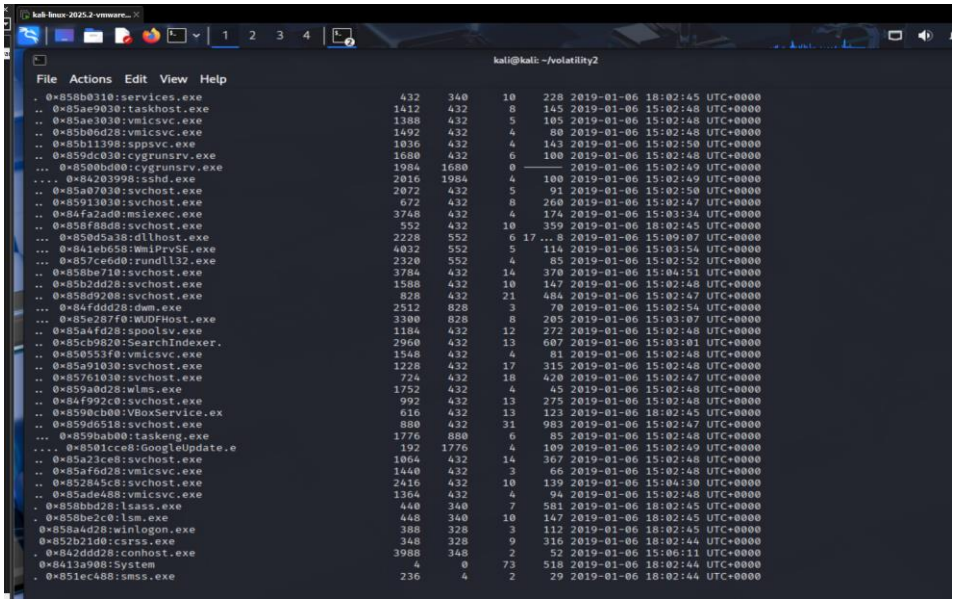




To further examine the execution context, the pstree plugin was used to reconstruct the parent-child relationships between processes. The resulting tree showed several expected Windows process relationships, including wininit.exe (PID 340) as the parent of

services.exe (PID 432), and explorer.exe (PID 2524) as the parent of the command prompt, FTK Imager, and the main Chrome process (PID 2388). The process tree also displayed an OpenSSH-related chain in which cygrunsrv.exe (PID 1680) was associated with another cygrunsrv.exe process (PID 1984), which in turn was associated with sshd.exe (PID 2016). The second cygrunsrv.exe process was listed with zero threads and no displayed handle count, making it a point for further examination rather than conclusive evidence of suspicious activity. Several Chrome child processes were also identified, consistent with the browser's multi-process architecture.



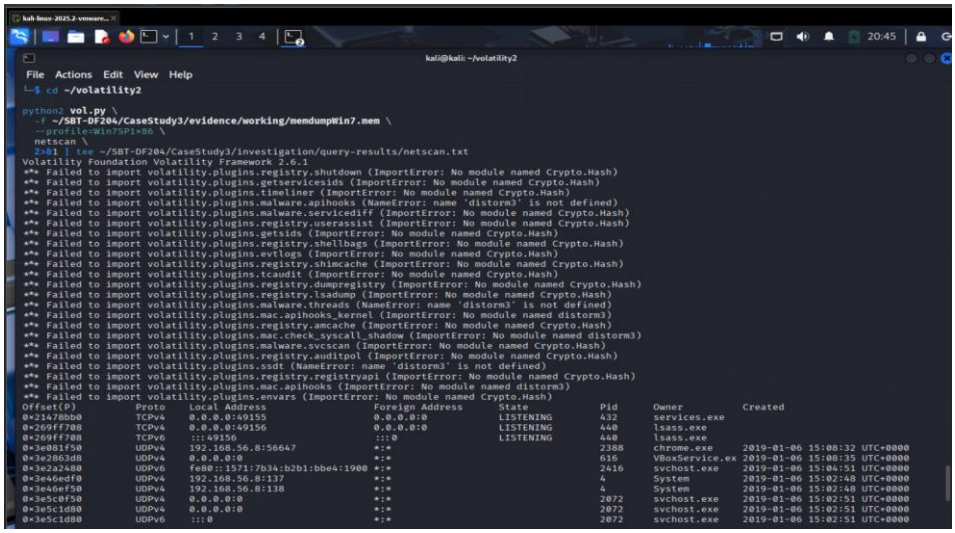


Overall, these analyses provided an initial view of the processes present in memory, their recorded command lines, and their parent-child relationships. The results identify areas for further investigation, particularly the command prompt, the FTK Imager process, and the OpenSSH-related service chain. The psscan output remains inconclusive because it returned no process entries, while the other two plugins successfully produced useful process information. The findings will therefore require correlation with additional memory artefacts before any determination can be made regarding potentially suspicious activity. The dependency warnings were documented as technical limitations, and no conclusion of compromise or unauthorised access is drawn from these results alone.

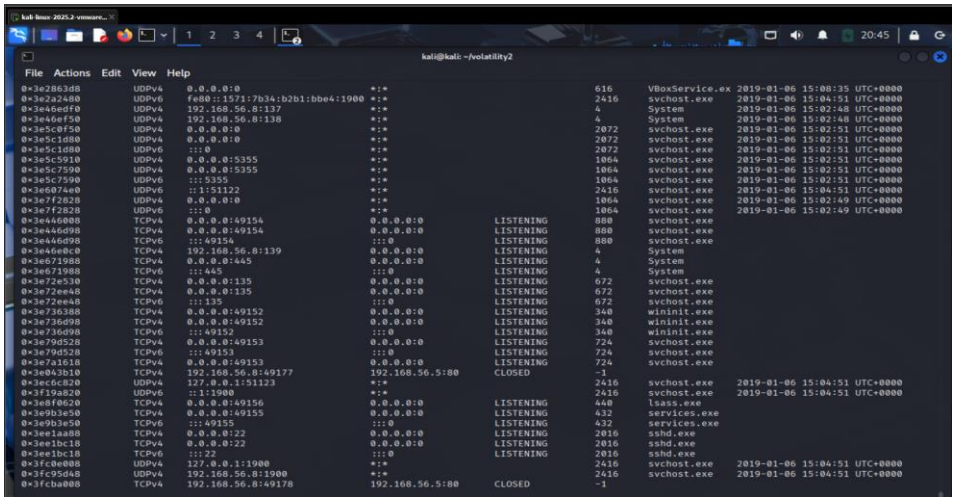
Network Artefact Analysis Using Volatility

Network artefact analysis was conducted using the netscan plugin of Volatility Framework 2.6.1 against the working copy of the memory image, memdumpWin7.mem, using the Win7SP1x86 profile. The objective was to identify network endpoints, listening services, connection states, and the processes associated with network activity that could be relevant to the investigation of a suspected unauthorised file transfer. The command

output was preserved in investigation/query-results/netscan.txt for subsequent examination and correlation with other evidence.



The output identified several TCP services listening on the workstation, including OpenSSH on port 22, associated with sshd.exe (PID 2016); SMB-related ports 139 and 445, associated with the System process (PID 4); and RPC-related port 135, associated with svchost.exe (PID 672). Additional listening ports included 49152, 49153, 49154, 49155, and 49156, which were associated with Windows system processes and services. These entries establish that the memory image contained network-listening artefacts at the time represented by the captured evidence. In particular, the OpenSSH service listening on port 22 is relevant because the earlier process analysis also identified sshd.exe (PID 2016) and related cygrunsrv.exe processes. However, the presence of a listening service does not, by itself, establish that an external party connected to the workstation or that files were transferred.



Two TCP records are particularly relevant to further investigation. The first, at offset 0x3e043b10, records a connection from local address 192.168.56.8:49177 to foreign address 192.168.56.5:80, with a state of CLOSED and no associated process ID displayed. The second, at offset 0x3fcba008, records a connection from 192.168.56.8:49178 to the same foreign address and port, also in the CLOSED state with no associated process ID. These records show that connection artefacts involving the two addresses were recovered from memory. However, because both connections were marked as closed and the output did not identify an owning process, the records do not establish which application initiated them, what data may have been exchanged, or whether a file transfer occurred. The remote address and port should therefore be correlated with other recovered artefacts rather than treated as proof of a transfer.

The scan also identified a UDP artefact associated with chrome.exe (PID 2388), using local address 192.168.56.8:56647, with a recorded creation time of 6 January 2019 at 15:08:32 UTC. Other UDP records were associated with Windows services, the System process, VBoxService.exe, and svchost.exe. These entries provide additional context about network activity and the virtualised environment represented in the image. Nevertheless, a UDP endpoint does not necessarily represent a completed

communication session, and the presence of a browser process does not identify the particular website, document, or file involved.

The output included warnings indicating that several Volatility plugins could not be imported because the Python dependencies Crypto.Hash and distorm3 were unavailable. Despite these warnings, the netscan plugin produced network records. The warnings should be documented as environmental limitations, although they do not automatically invalidate the records returned by this particular plugin. The recovered network artefacts should be interpreted alongside the process list, process tree, command history, browser-related evidence, removable-media records, and user-profile artefacts before drawing conclusions about the suspected transfer.

In conclusion, the network scan successfully recovered listening-service records and two closed TCP connection records involving 192.168.56.8 and 192.168.56.5 on port 80. The OpenSSH listener associated with PID 2016 and the two closed TCP records are relevant investigative leads. However, the available output does not establish a successful file transfer, identify a transferred file, or attribute the activity to a particular individual. Further correlation with command history, USB and drive-mapping records, user-profile artefacts, and ShellBag results is required before reconstructing a defensible sequence of events.

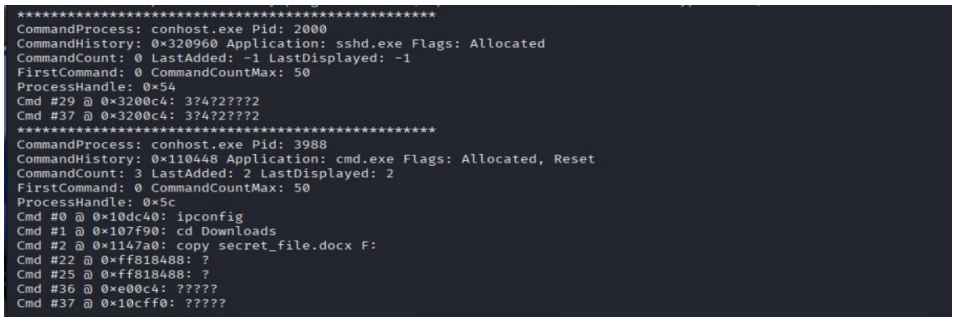
Evidence log entry

Field	**Entry**
Evidence ID	E-next (assign the next unused ID)
Tool and plugin	Volatility 2.6.1 — netscan
Profile	Win7SP1x86
Source	memdumpWin7.mem — working copy
Output file	investigation/query-results/netscan.txt
Relevant records	0x3e043b10, 0x3fcba008, 0x3ee1aa88, 0x3ee1bc18

Screenshot	Use the next available figure number
Interpretation	Network services and closed TCP connection artefacts identified; no completed transfer established

Command History Analysis Using Volatility

Command history analysis was performed using the cmdscan plugin of Volatility Framework 2.6.1 against the working copy of the memory image, memdumpWin7.mem, with the Win7SP1x86 profile. The purpose of this examination was to recover command-prompt history that could provide evidence of user activity, file-system navigation, and a possible transfer of a confidential document. The command output was saved to investigation/query-results/cmdscan.txt for preservation and correlation with other artefacts.



The output identified a command history associated with conhost.exe (PID 3988), with the recorded application identified as cmd.exe. The history contained three numbered commands: ipconfig, cd Downloads, and copy secret_file.docx F:. The ipconfig command is commonly used to display network configuration information, while cd Downloads indicates navigation to a directory named Downloads. The subsequent command, copy secret_file.docx F:, is particularly relevant to the investigation because it specifies a file named secret_file.docx as the source and F: as the destination. Read in sequence, these commands are consistent with a possible workflow in which a user examined network

configuration, navigated to a Downloads directory, and then attempted to copy a document to a drive identified by the letter F.

The recovered command history provides a significant investigative lead because the specified filename and destination drive are consistent with the case scenario involving a suspected unauthorised file transfer. However, the cmdscan output establishes only that the command string was recovered from memory. It does not establish that secret_file.docx existed in the current directory, that the F: drive was available or mapped to removable media, or that the copy operation completed successfully. The output also does not identify the contents or classification of the file, the physical identity of any destination device, or the individual who entered the command. These questions require corroboration through console records, removable-media registry artefacts, drive mappings, user-profile evidence, and File Explorer traces.

A second command-history structure was associated with conhost.exe (PID 2000), with the application identified as sshd.exe. This structure contained unusual character sequences rather than clearly interpretable commands. Its metadata reported a command count of zero and no recorded last-added or last-displayed command index. Consequently, the available output does not provide a reliable basis for reconstructing command activity from this structure. Although its association with the SSH service is worth retaining as an investigative lead, the recovered strings do not independently establish remote command execution or file transfer.

The output also contained additional entries represented by question marks or non-readable characters in the command history associated with PID 3988. These entries cannot be reliably interpreted from the displayed output and should not be treated as identifiable commands. Their presence illustrates the limitations of recovering console history from volatile memory, where structures may contain residual, overwritten, or partially recoverable data.

In conclusion, the cmdscan analysis recovered a sequence of three readable commands associated with cmd.exe, including copy secret_file.docx F:. This is material evidence supporting the possibility of a file-copy attempt to a destination identified as F:. Nevertheless, the command history alone cannot confirm successful execution, establish

that F: represented a USB device, or attribute the activity to a specific person. The finding should be correlated with the consoles output, USB-related registry artefacts, MountedDevices, MountPoints2, ShellBags, and the other process and network evidence before a final conclusion is reached.

Evidence log entry

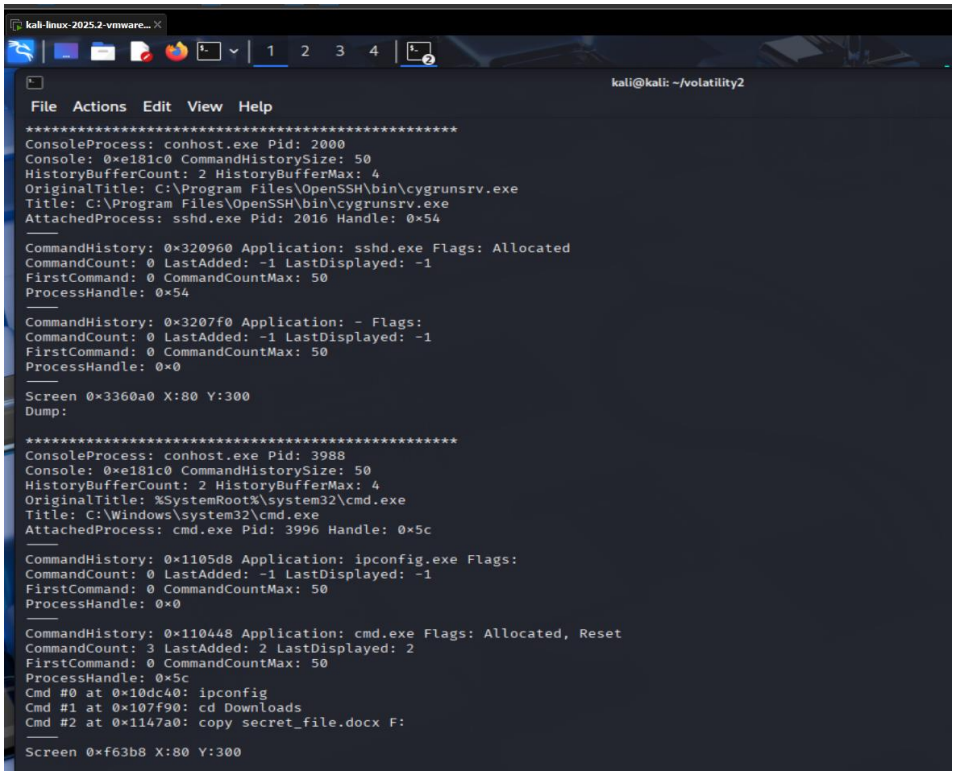
Field	Entry
Evidence ID	Next unused ID
Tool and plugin	Volatility 2.6.1 — cmdscan
Profile	Win7SP1x86
Source	memdumpWin7.mem — working copy
Output	investigation/query-results/cmdscan.txt
Key record	conhost.exe PID 3988; application cmd.exe; history address 0x110448
Material command	copy secret_file.docx F:
Screenshot	Next available figure number
Finding	Recovered command string consistent with a possible file-copy attempt; successful transfer not established

Console Analysis Using Volatility

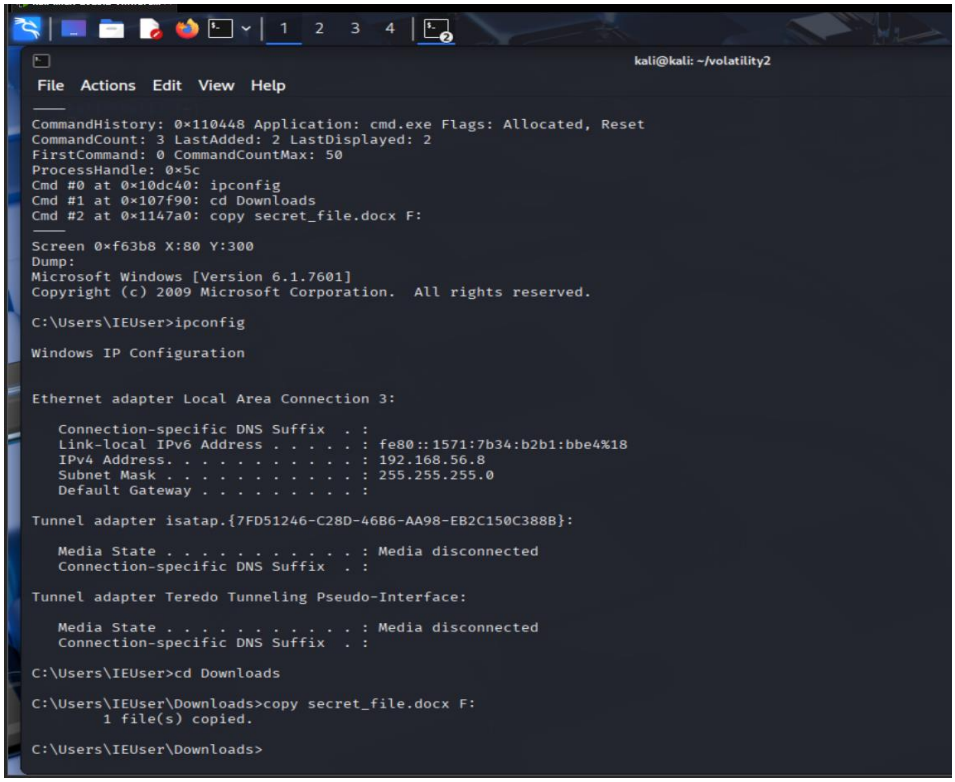
Console analysis was conducted using the consoles plugin of Volatility Framework 2.6.1 against the working copy of the memory image, memdumpWin7.mem, using the Win7SP1x86 profile. The objective was to examine recovered console structures, command history, and screen-buffer contents to determine whether additional evidence could corroborate the possible file-transfer activity identified during the preceding

command-history analysis. The output was saved to investigation/query-results/consoles.txt for preservation and subsequent correlation with other forensic artefacts.

The output identified a console associated with conhost.exe (PID 3988), attached to cmd.exe (PID 3996). The console title was recorded as C:\Windows\system32\cmd.exe, and the recovered screen buffer contained a Windows command-prompt session operating under the directory C:\Users\IEUser\Downloads. The console output displayed the commands ipconfig, cd Downloads, and copy secret_file.docx F: in that order. The ipconfig output recorded the workstation's IPv4 address as 192.168.56.8, with a subnet mask of 255.255.255.0 and no default gateway displayed. This network configuration provides contextual information that can be compared with the network artefacts recovered during the netscan analysis.



The most significant finding was the output immediately following the command copy secret_file.docx F:. The console screen buffer displayed the message 1 file(s) copied. followed by the prompt C:\Users\IEUser\Downloads>. This is stronger evidence of a reported successful copy operation than the command string recovered by cmdscan alone. The sequence indicates that the Windows command interpreter displayed a success message after the copy command was entered. However, the console output does not independently establish the physical identity of the destination, whether F: was a removable storage device, or whether the resulting file remained intact and accessible after the operation. These matters require further examination of USB device records, drive mappings, and related user-profile artefacts.



The console records also identified a separate structure associated with conhost.exe (PID 2000), attached to sshd.exe (PID 2016), with the title C:\Program

Files\OpenSSH\bin\cygrunsrv.exe. Its recovered command-history structures reported zero commands, and the displayed screen buffer contained no readable command output. Although this confirms that an OpenSSH-related console structure was present in the captured memory, it does not demonstrate that the file-copy command was executed through SSH or that a remote connection was responsible for the observed activity. The file-copy command was recovered from the separate console attached to cmd.exe (PID 3996).

The console analysis therefore corroborates the principal finding from cmdscan: the command copy secret_file.docx F: was present in the command history, and the corresponding console screen buffer contained the message 1 file(s) copied. This evidence supports, with greater confidence, the interpretation that the command interpreter reported a successful file-copy operation. Nevertheless, the destination device, the sensitivity or contents of the document, and the identity of the person responsible remain unverified. The finding must be correlated with USBSTOR, DeviceClasses, MountedDevices, MountPoints2, ShellBags, and relevant account artefacts to determine whether the available evidence supports a removable-media transfer and to establish the limits of any attribution.

The consoles plugin returned useful console and screen-buffer information despite the previously observed warnings concerning missing Python dependencies. These warnings were documented as environmental limitations. The recovered records provide a material lead for the investigation but should be interpreted together with the remaining artefacts before a final conclusion is reached.

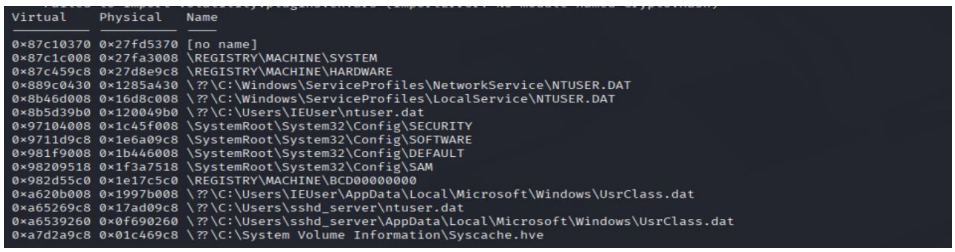
Evidence log entry

Field	**Entry**
Tool and plugin	Volatility 2.6.1 - consoles
Profile	Win7SP1x86
Output file	investigation/query-results/consoles.txt

Console process	cmd.exe, PID 3996
Console host	conhost.exe, PID 3988
Relevant record	Command history address 0x110448; screen buffer 0xf63b8
Material observation	copy secret_file.docx F: followed by 1 file(s) copied.
Screenshot	Use the next available figure number
Interpretation	Console reports a successful copy; destination device and human attribution remain unverified

Registry Hive Identification

The Volatility Framework 2.6.1 hivelist plugin was executed against the working copy of the memory image using the Win7SP1x86 profile. The command successfully identified multiple registry hives resident in memory, including the SYSTEM, SOFTWARE, SECURITY, SAM and DEFAULT hives, as well as user-specific registry files associated with the IEUser and sshd_server profiles. The SYSTEM hive was identified at virtual offset 0x87c1c008, while the SOFTWARE hive was identified at virtual offset 0x9711d9c8. The IEUser profile's ntuser.dat hive was identified at virtual offset 0x8b5d39b0, and its UsrClass.dat hive at 0xa620b008. These results provide the locations required for subsequent registry analysis of storage-device configuration, drive mappings and Explorer artefacts.

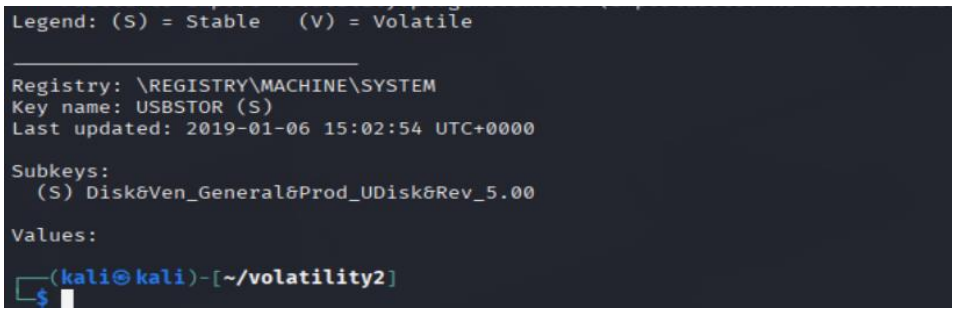


```
Virtual    Physical   Name
---------- ---------- ----
0x87c10370 0x27fd5370 [no name]
0x87c1c008 0x27fa3008 \REGISTRY\MACHINE\SYSTEM
0x87c459c8 0x27d8e9c8 \REGISTRY\MACHINE\HARDWARE
0x889c0430 0x1285a430 \??\C:\Windows\ServiceProfiles\NetworkService\NTUSER.DAT
0x8b46d008 0x16d8c008 \??\C:\Windows\ServiceProfiles\LocalService\NTUSER.DAT
0x8b5d39b0 0x120049b0 \??\C:\Users\IEUser\ntuser.dat
0x97104008 0x1c45f008 \SystemRoot\System32\Config\SECURITY
0x9711d9c8 0x1e6a09c8 \SystemRoot\System32\Config\SOFTWARE
0x981f9008 0x1b446008 \SystemRoot\System32\Config\DEFAULT
0x98209518 0x1f3a7518 \SystemRoot\System32\Config\SAM
0x982d55c0 0x1e17c5c0 \REGISTRY\MACHINE\BCD00000000
0xa620b008 0x1997b008 \??\C:\Users\IEUser\AppData\Local\Microsoft\Windows\UsrClass.dat
0xa65269c8 0x17ad09c8 \??\C:\Users\sshd_server\ntuser.dat
0xa6539260 0x0f690260 \??\C:\Users\sshd_server\AppData\Local\Microsoft\Windows\UsrClass.dat
0xa7d2a9c8 0x01c469c8 \??\C:\System Volume Information\Syscache.hve
```

The IEUser profile was prioritised because the previously recovered command-console evidence showed a command attempting to copy secret_file.docx from the Downloads directory to drive F:, with the console displaying 1 file(s) copied. The hive listing itself does not establish that drive F: was a USB device or identify the person who performed the command; these questions require corroboration from specific registry keys and other evidence. Volatility also reported that several plugins could not be imported because the Python dependencies Crypto.Hash and distorm3 were unavailable. Although hivelist produced results, these dependency limitations will be considered when selecting and interpreting subsequent plugins.

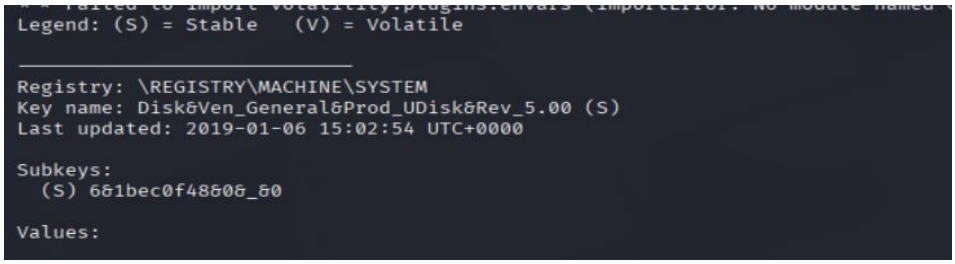
USB Mass-Storage Registry Examination

The Volatility printkey plugin was used to examine the ControlSet001\Enum\USBSTOR registry key in the SYSTEM hive, identified at virtual offset 0x87c1c008. The output showed a USB storage device subkey named Disk&Ven_General&Prod_UDisk&Rev_5.00, with a registry last-updated timestamp of 6 January 2019 at 15:02:54 UTC. The device identifier is consistent with a USB mass-storage device described by Windows as a General UDisk device with revision 5.00. This artefact is relevant to the investigation because the recovered command-console buffer showed the command copy secret_file.docx F: followed by the message 1 file(s) copied. However, the USBSTOR entry alone does not establish the device's unique identity, its assigned drive letter, or a direct relationship between the device and the reported copy operation. Further examination of the device's subkeys and the MountedDevices registry data is required before drawing a conclusion about whether this device corresponds to drive F:. The timestamp represents the registry key's recorded last-update time and should not, without corroboration, be treated as the exact time the file was copied.



USB Device Instance Identification

Further examination of the USBSTOR registry entry identified the device-instance subkey 6&1bec0f48&0&_&0 beneath Disk&Ven_General&Prod_UDisk&Rev_5.00. This establishes that the recovered SYSTEM registry data contains a specific instance entry associated with the previously identified General UDisk mass-storage device. The parent key's last-updated timestamp remained 6 January 2019 at 15:02:54 UTC.

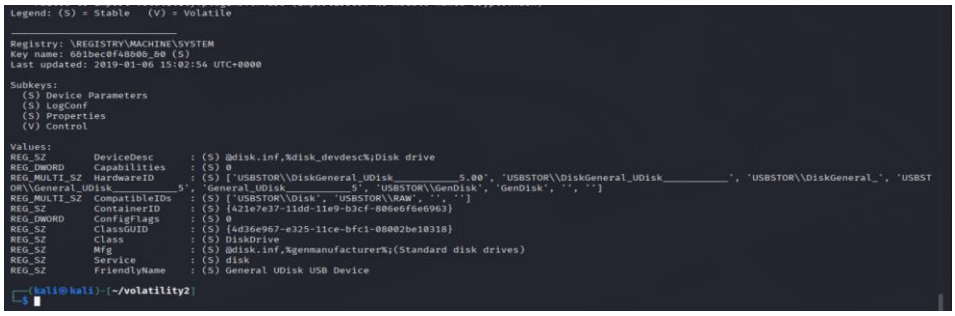


The instance subkey provides a more specific path for further examination of the device's recorded properties. However, the identifier should not be interpreted as a verified hardware serial number without inspecting the instance's values and corroborating the result with other registry artefacts. At this stage, the evidence supports the presence of a USB storage device entry in the recovered registry, but does not yet establish that it was assigned drive letter F: or that it received the file secret_file.docx.

USB Storage Device Properties

The device-instance key ControlSet001\Enum\USBSTOR\Disk&Ven_General&Prod_UDisk&Rev_5.00\6&1bec0f48&0&_&0 was examined using Volatility's printkey plugin against the recovered

SYSTEM hive. The key contained the FriendlyName value General UDisk USB Device, identified the device class as DiskDrive, and listed disk as the associated service. The HardwareID and CompatibleIDs values included identifiers consistent with a USB mass-storage disk, while the ContainerID was recorded as {421e7e37-11dd-11e9-b3cf-806e6f6e6963}.



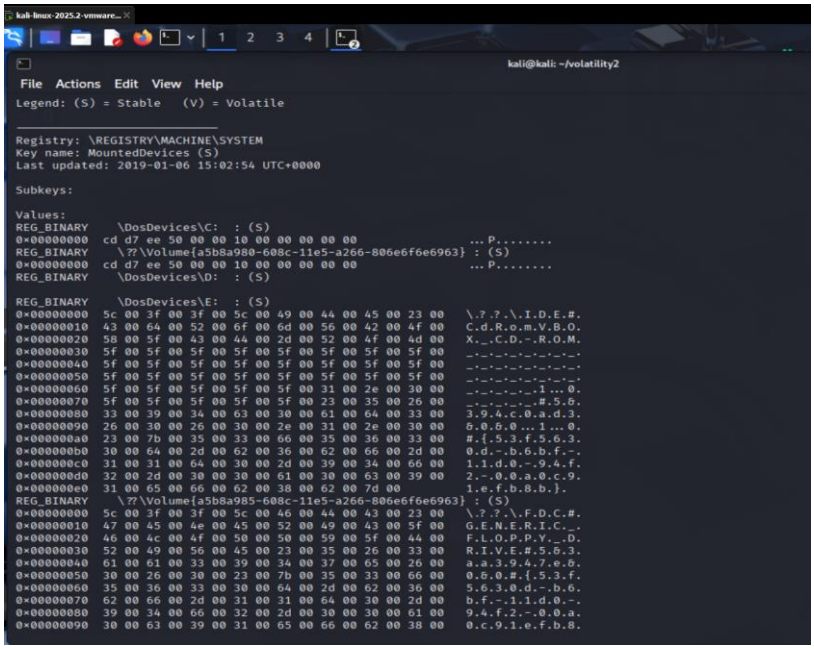
The registry key's last-updated timestamp was 6 January 2019 at 15:02:54 UTC. These properties strengthen the finding that Windows had a registry entry for a General UDisk USB storage device. However, the values shown do not establish its assigned drive letter or demonstrate that secret_file.docx was transferred to it. The next stage is to examine Windows drive mappings and correlate them with this device entry and the previously recovered command-console output.

Property	**Recovered value**
Friendly name	General UDisk USB Device
Device class	DiskDrive
Service	disk
Container ID	{421e7e37-11dd-11e9-b3cf-806e6f6e6963}
Capabilities	0
Last-updated timestamp	2019-01-06 15:02:54 UTC

The hardware identifiers are consistent with a USB mass-storage disk. The ContainerID is a correlation value we can preserve for later comparison, but it does not independently identify a person or prove a file transfer.

Drive-Letter Mapping and USB Device Correlation

The MountedDevices key in the recovered SYSTEM registry hive was examined using Volatility 2.6.1. The output contained a \DosDevices\F: value whose binary data decoded to a device path identifying USBSTOR\Disk&Ven_General&Prod_UDisk&Rev_5.00#6&1bec0f48&0&_&0, matching the model and instance identifier previously recovered from the USBSTOR registry key. A corresponding volume entry, \??\Volume{421e7e52-11dd-11e9-b3cf-08002709e15d}, contained the same USBSTOR device path. This establishes a direct registry-level correlation between drive letter F: and the identified General UDisk USB mass-storage device. When considered alongside the recovered command-console buffer, which records copy secret_file.docx F: followed by 1 file(s) copied., the evidence supports the conclusion that the Windows command interpreter reported copying the named document to a drive mapped to the identified USB storage device. This correlation strengthens the suspected file-transfer finding. Nevertheless, the available artefacts do not independently establish the destination file's present existence or integrity, reveal its contents, or identify the individual who operated the computer. The MountedDevices key's last-updated timestamp of 6 January 2019 at 15:02:54 UTC should not be treated as the exact time of the copy operation.



```
kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/volatility2
File Actions Edit View Help
Legend: (S) = Stable   (V) = Volatile

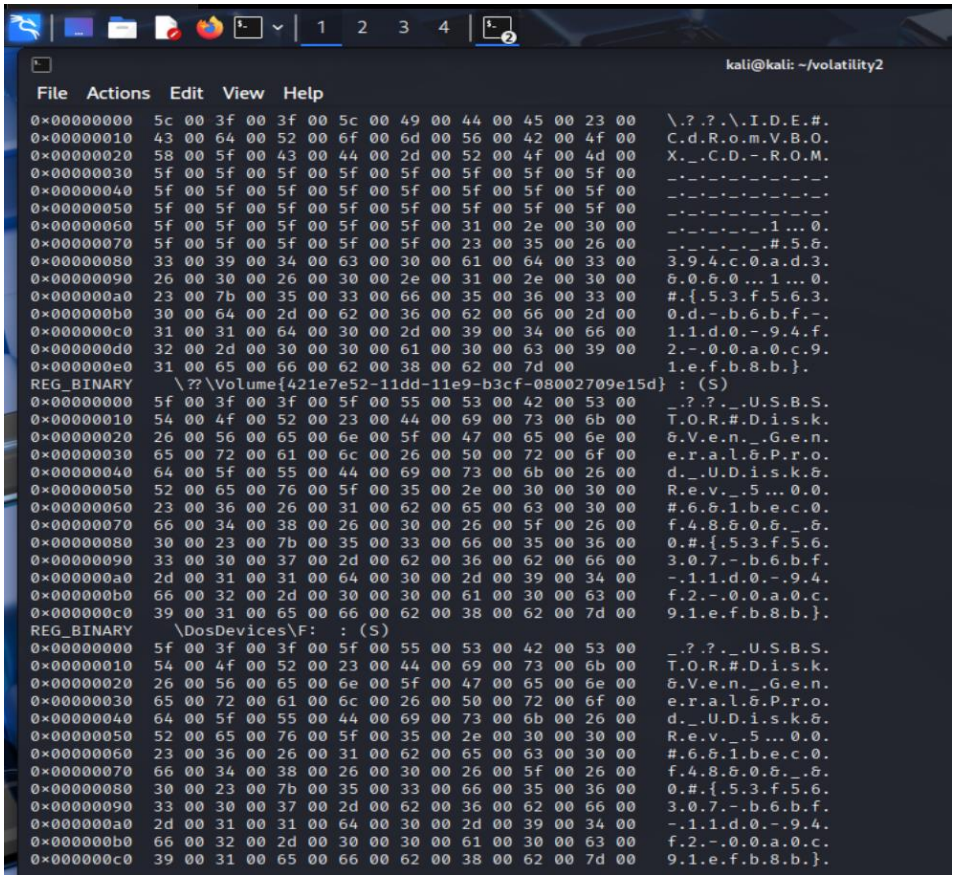
----------
Registry: \REGISTRY\MACHINE\SYSTEM
Key name: MountedDevices (S)
Last updated: 2019-01-06 15:02:54 UTC+0000

Subkeys:

Values:
REG_BINARY    \DosDevices\C:  : (S)
0x00000000  cd d7 ee 50 00 00 10 00 00 00 00 00                ...P........
REG_BINARY    \??\Volume{a5b8a980-608c-11e5-a266-806e6f6e6963} : (S)
0x00000000  cd d7 ee 50 00 00 10 00 00 00 00 00                ...P........
REG_BINARY    \DosDevices\D:  : (S)

REG_BINARY    \DosDevices\E:  : (S)
0x00000000  5c 00 3f 00 3f 00 5c 00 49 00 44 00 45 00 23 00   \.?.?.\.I.D.E.#.
0x00000010  43 00 64 00 52 00 6f 00 6d 00 56 00 42 00 4f 00   C.d.R.o.m.V.B.O.
0x00000020  58 00 5f 00 43 00 44 00 2d 00 52 00 4f 00 4d 00   X._.C.D.-.R.O.M.
0x00000030  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000040  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000050  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000060  5f 00 5f 00 5f 00 5f 00 5f 00 31 00 2e 00 30 00   _._._._._._1...0.
0x00000070  5f 00 5f 00 5f 00 5f 00 5f 00 23 00 35 00 26 00   _._._._._.#.5.&.
0x00000080  33 00 39 00 34 00 63 00 30 00 61 00 64 00 33 00   3.9.4.c.0.a.d.3.
0x00000090  26 00 30 00 26 00 30 00 2e 00 31 00 2e 00 30 00   &.0.&.0...1...0.
0x000000a0  23 00 7b 00 35 00 33 00 66 00 35 00 36 00 33 00   #.{.5.3.f.5.6.3.
0x000000b0  30 00 64 00 2d 00 62 00 36 00 62 00 66 00 2d 00   0.d.-.b.6.b.f.-.
0x000000c0  31 00 31 00 64 00 30 00 2d 00 39 00 34 00 66 00   1.1.d.0.-.9.4.f.
0x000000d0  32 00 2d 00 30 00 30 00 61 00 30 00 63 00 39 00   2.-.0.0.a.0.c.9.
0x000000e0  31 00 65 00 66 00 62 00 38 00 62 00 7d 00         1.e.f.b.8.b.}.
REG_BINARY    \??\Volume{a5b8a985-608c-11e5-a266-806e6f6e6963} : (S)
0x00000000  5c 00 3f 00 3f 00 5c 00 46 00 44 00 43 00 23 00   \.?.?.\.F.D.C.#.
0x00000010  47 00 45 00 4e 00 45 00 52 00 49 00 43 00 5f 00   G.E.N.E.R.I.C._.
0x00000020  46 00 4c 00 4f 00 50 00 50 00 59 00 5f 00 44 00   F.L.O.P.P.Y._.D.
0x00000030  52 00 49 00 56 00 45 00 23 00 35 00 26 00 33 00   R.I.V.E.#.5.&.3.
0x00000040  61 00 61 00 33 00 39 00 34 00 37 00 65 00 26 00   a.a.3.9.4.7.e.&.
0x00000050  30 00 26 00 30 00 23 00 7b 00 35 00 33 00 66 00   0.&.0.#.{.5.3.f.
0x00000060  35 00 36 00 33 00 30 00 64 00 2d 00 62 00 36 00   5.6.3.0.d.-.b.6.
0x00000070  62 00 66 00 2d 00 31 00 31 00 64 00 30 00 2d 00   b.f.-.1.1.d.0.-.
0x00000080  39 00 34 00 66 00 32 00 2d 00 30 00 30 00 61 00   9.4.f.2.-.0.0.a.
0x00000090  30 00 63 00 39 00 31 00 65 00 66 00 62 00 38 00   0.c.9.1.e.f.b.8.
```

```
File Actions Edit View Help
0x000000e0  31 00 65 00 66 00 62 00 38 00 62 00 7d 00         1.e.f.b.8.b.}.
REG_BINARY    \??\Volume{a5b8a985-608c-11e5-a266-806e6f6e6963} : (S)
0x00000000  5c 00 3f 00 3f 00 5c 00 46 00 44 00 43 00 23 00   \.?.?.\.F.D.C.#.
0x00000010  47 00 45 00 4e 00 45 00 52 00 49 00 43 00 5f 00   G.E.N.E.R.I.C._.
0x00000020  46 00 4c 00 4f 00 50 00 50 00 59 00 5f 00 44 00   F.L.O.P.P.Y._.D.
0x00000030  52 00 49 00 56 00 45 00 23 00 35 00 26 00 33 00   R.I.V.E.#.5.&.3.
0x00000040  61 00 61 00 33 00 39 00 34 00 37 00 65 00 26 00   a.a.3.9.4.7.e.&.
0x00000050  30 00 26 00 30 00 23 00 7b 00 35 00 33 00 66 00   0.&.0.#.{.5.3.f.
0x00000060  35 00 36 00 33 00 30 00 64 00 2d 00 62 00 36 00   5.6.3.0.d.-.b.6.
0x00000070  62 00 66 00 2d 00 31 00 31 00 64 00 30 00 2d 00   b.f.-.1.1.d.0.-.
0x00000080  39 00 34 00 66 00 32 00 2d 00 30 00 30 00 61 00   9.4.f.2.-.0.0.a.
0x00000090  30 00 63 00 39 00 31 00 65 00 66 00 62 00 38 00   0.c.9.1.e.f.b.8.
0x000000a0  62 00 7d 00                                       b.}.
REG_BINARY    \??\Volume{762f4ebc-60ea-11e5-83af-806e6f6e6963} : (S)
0x00000000  5c 00 3f 00 3f 00 5c 00 49 00 44 00 45 00 23 00   \.?.?.\.I.D.E.#.
0x00000010  43 00 64 00 52 00 6f 00 6d 00 56 00 42 00 4f 00   C.d.R.o.m.V.B.O.
0x00000020  58 00 5f 00 43 00 44 00 2d 00 52 00 4f 00 4d 00   X._.C.D.-.R.O.M.
0x00000030  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000040  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000050  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000060  5f 00 5f 00 5f 00 5f 00 5f 00 31 00 2e 00 30 00   _._._._._._1...0.
0x00000070  5f 00 5f 00 5f 00 5f 00 5f 00 23 00 35 00 26 00   _._._._._.#.5.&.
0x00000080  33 00 39 00 34 00 63 00 30 00 61 00 64 00 33 00   3.9.4.c.0.a.d.3.
0x00000090  26 00 30 00 26 00 30 00 2e 00 30 00 2e 00 30 00   &.0.&.0...0...0.
0x000000a0  23 00 7b 00 35 00 33 00 66 00 35 00 36 00 33 00   #.{.5.3.f.5.6.3.
0x000000b0  30 00 64 00 2d 00 62 00 36 00 62 00 66 00 2d 00   0.d.-.b.6.b.f.-.
0x000000c0  31 00 31 00 64 00 30 00 2d 00 39 00 34 00 66 00   1.1.d.0.-.9.4.f.
0x000000d0  32 00 2d 00 30 00 30 00 61 00 30 00 63 00 39 00   2.-.0.0.a.0.c.9.
0x000000e0  31 00 65 00 66 00 62 00 38 00 62 00 7d 00         1.e.f.b.8.b.}.
REG_BINARY    \??\Volume{8358fe6d-60fa-11e5-bb4a-806e6f6e6963} : (S)
0x00000000  5c 00 3f 00 3f 00 5c 00 49 00 44 00 45 00 23 00   \.?.?.\.I.D.E.#.
0x00000010  43 00 64 00 52 00 6f 00 6d 00 56 00 42 00 4f 00   C.d.R.o.m.V.B.O.
0x00000020  58 00 5f 00 43 00 44 00 2d 00 52 00 4f 00 4d 00   X._.C.D.-.R.O.M.
0x00000030  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000040  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000050  5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00 5f 00   _._._._._._._._.
0x00000060  5f 00 5f 00 5f 00 5f 00 5f 00 31 00 2e 00 30 00   _._._._._._1...0.
0x00000070  5f 00 5f 00 5f 00 5f 00 5f 00 23 00 35 00 26 00   _._._._._.#.5.&.
0x00000080  33 00 39 00 34 00 63 00 30 00 61 00 64 00 33 00   3.9.4.c.0.a.d.3.
0x00000090  26 00 30 00 26 00 30 00 2e 00 31 00 2e 00 30 00   &.0.&.0...1...0.
0x000000a0  23 00 7b 00 35 00 33 00 66 00 35 00 36 00 33 00   #.{.5.3.f.5.6.3.
0x000000b0  30 00 64 00 2d 00 62 00 36 00 62 00 66 00 2d 00   0.d.-.b.6.b.f.-.
0x000000c0  31 00 31 00 64 00 30 00 2d 00 39 00 34 00 66 00   1.1.d.0.-.9.4.f.
```



The MountedDevices registry contains a \DosDevices\F: entry that matches the USBSTOR device identifier recovered in the previous steps.

The correlation is direct at the registry-data level:

- The USBSTOR device entry identifies Disk&Ven_General&Prod_UDisk&Rev_5.00, instance 6&1bec0f48&0&_&0.
- The \DosDevices\F: value contains the corresponding USBSTOR device path, including that same model and instance identifier.

- The associated volume entry, \??\Volume{421e7e52-11dd-11e9-b3cf-08002709e15d}, also contains the matching device path.
- The recovered command-console output records copy secret_file.docx F: and reports 1 file(s) copied.

Taken together, these are mutually corroborating artefacts: the console records a reported copy operation to F:, and the registry associates F: with the same General UDisk device identified under USBSTOR.

This is substantially stronger than merely finding a USB device in the registry. However, the console message is evidence that Windows reported a successful copy operation; it does not independently verify the destination file's contents or continued existence.

User Profile MountPoints2 Examination

The MountPoints2 registry key was examined in the IEUser profile hive, identified as C:\Users\IEUser\ntuser.dat at virtual offset 0x8b5d39b0. The recovered key contained subkeys named F and {421e7e52-11dd-11e9-b3cf-08002709e15d}, alongside several other volume-GUID entries. The GUID {421e7e52-11dd-11e9-b3cf-08002709e15d} matches the volume entry associated with the General UDisk USB storage device in the SYSTEM hive's MountedDevices data. The MountPoints2 key's last-updated timestamp was 6 January 2019 at 15:03:07 UTC.

```
Legend: (S) = Stable   (V) = Volatile

__________________________

Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: MountPoints2 (S)
Last updated: 2019-01-06 15:03:07 UTC+0000

Subkeys:
  (S) CPC
  (S) F
  (S) {421e7e52-11dd-11e9-b3cf-08002709e15d}
  (S) {762f4ebc-60ea-11e5-83af-806e6f6e6963}
  (S) {8358fe6d-60fa-11e5-bb4a-806e6f6e6963}
  (S) {a5b8a980-608c-11e5-a266-806e6f6e6963}
  (S) {a5b8a983-608c-11e5-a266-806e6f6e6963}

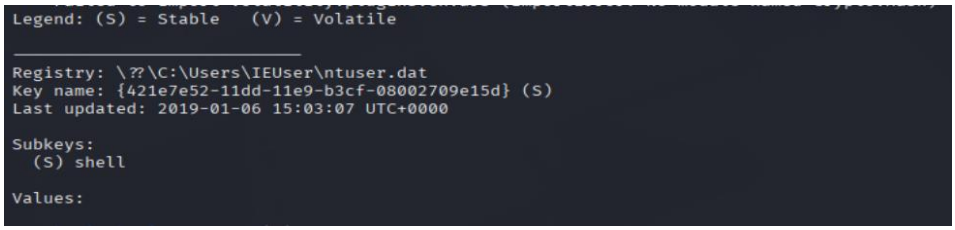
Values:
```

The presence of both the F entry and the matching volume GUID in the IEUser profile provides additional corroboration that the volume was represented in the user's Explorer-related mount-point registry data. When considered alongside the matching USBSTOR

device instance, the F: drive mapping, and the console's reported copy operation, this finding strengthens the hypothesis that the identified USB device was the destination specified by the command. However, the MountPoints2 listing alone does not prove that the file was successfully written and retained on the device, and its key timestamp should not be treated as an exact event time without further corroboration.

Examination of the Matching Mount Point Entry

The MountPoints2 subkey associated with volume GUID {421e7e52-11dd-11e9-b3cf-08002709e15d} was examined within the IEUser registry hive. The key was present and contained a subkey named shell; no direct values were displayed at the inspected level. The key's last-updated timestamp was 6 January 2019 at 15:03:07 UTC.



This GUID matches the volume entry previously correlated with the General UDisk device through the SYSTEM hive's MountedDevices data. The result therefore confirms that the matching volume had a corresponding Explorer-related mount-point entry in the IEUser profile. The presence of the shell subkey warrants further inspection, but this output alone does not establish what actions, if any, were performed through that subkey. It also does not independently prove the successful persistence of secret_file.docx on the device.

Explorer Shell and AutoPlay Registry Examination

The shell subkey beneath the MountPoints2 entry for volume GUID {421e7e52-11dd-11e9-b3cf-08002709e15d} was examined in the IEUser profile hive. The recovered key contained a subkey named Autoplay and displayed an unnamed REG_SZ value as None.

```
Legend: (S) = Stable   (V) = Volatile

----------------------------------
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: shell (S)
Last updated: 2019-01-06 15:03:07 UTC+0000

Subkeys:
  (S) Autoplay

Values:
REG_SZ                        : (S) None
```

The key's last-updated timestamp was 6 January 2019 at 15:03:07 UTC. This artefact provides additional detail about the Explorer-related registry structure associated with the volume already correlated to the General UDisk device. However, the presence of an Autoplay subkey alone does not prove that AutoPlay was triggered, that a user browsed the volume, or that any particular file was opened. Its evidential value is therefore contextual and should be considered alongside the stronger correlations between the USBSTOR device instance, the F: drive mapping, and the recovered command-console copy operation.

Examination of the F Mount Point Entry

The MountPoints2\F subkey in the IEUser profile hive was examined using Volatility's printkey plugin. The key was recovered with a last-updated timestamp of 6 January 2019 at 15:03:06 UTC; however, no subkeys or values were displayed beneath it.

```
Legend: (S) = Stable   (V) = Volatile

----------------------------------
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: F (S)
Last updated: 2019-01-06 15:03:06 UTC+0000

Subkeys:

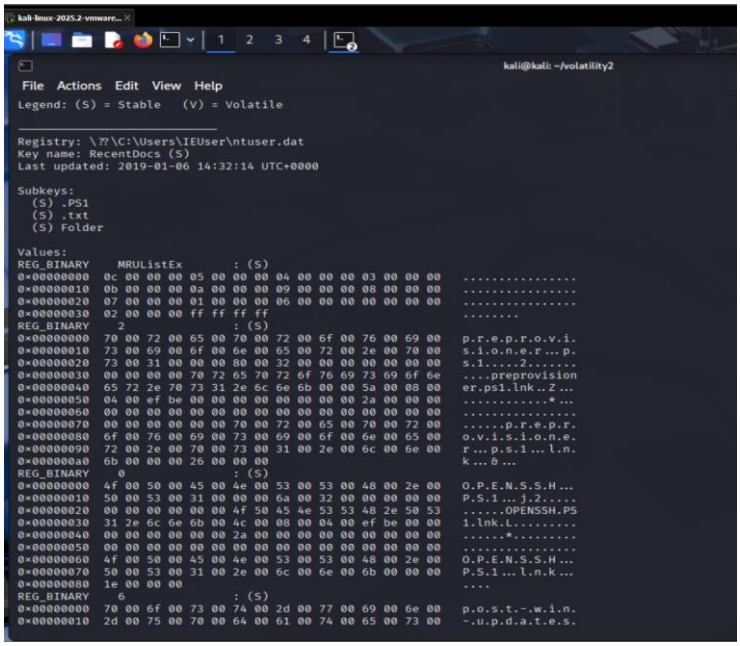
Values:
```

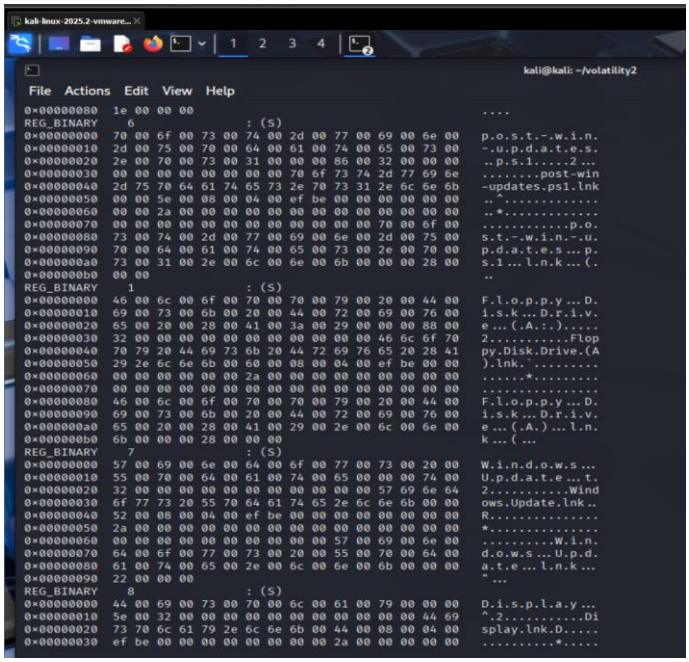
The finding confirms the presence of an F entry in the user's Explorer-related mount-point registry data but provides no additional device identifier or direct evidence of file activity. Accordingly, this result is treated as supporting context rather than independent proof of a transfer. The principal correlation remains the matching USBSTOR instance in the

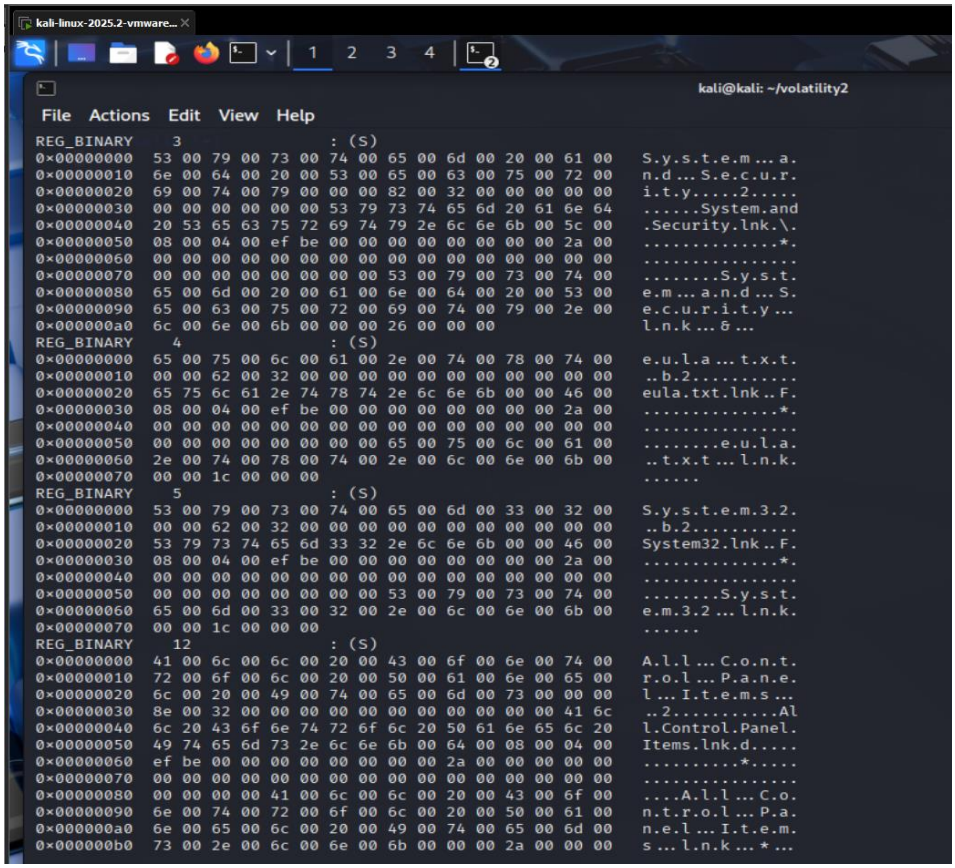
SYSTEM hive's MountedDevices mapping for drive F:, reinforced by the console output reporting that one file was copied to that drive.

Recent Documents Registry Examination

The Windows Registry key Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs was examined in the NTUSER.DAT hive belonging to the IEUser profile using the Volatility 2.6.1 printkey plugin. The key was reported as last updated on 6 January 2019 at 14:32:14 UTC. Its MRUListEx value contained an ordering of recorded entries, while the associated binary values included names such as OPENSSH.PS1, preprovisioner.ps1, post-win-updates.ps1, Readme.txt, and eula.txt, together with Windows interface shortcuts including System32, Display, and Network and Internet. These artefacts provide contextual evidence of items recorded in the profile's RecentDocs registry data. However, the displayed entries did not identify secret_file.docx. This absence does not establish that the document was never accessed, because RecentDocs is not a complete record of every file operation and entries may be absent, overwritten, or stored elsewhere. Likewise, the presence of PowerShell script names does not establish that the scripts were executed. The registry timestamp records the last update of the examined key and must not be interpreted as the access time of each listed item. The findings were therefore retained as contextual evidence and were not treated as proof of execution, document access, or file transfer.


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/volatility2
File Actions Edit View Help
Legend: (S) = Stable (V) = Volatile
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: RecentDocs (S)
Last updated: 2019-01-06 14:32:14 UTC+0000
Subkeys:
(S) .PS1
(S) .txt
(S) Folder
Values:
REG_BINARY MRUListEx : (S)
0x00000000 0c 00 00 00 05 00 00 00 04 00 00 00 03 00 00 00 ....
0x00000010 0b 00 00 00 0a 00 00 00 09 00 00 00 08 00 00 00 ....
0x00000020 07 00 00 00 01 00 00 00 06 00 00 00 00 00 00 00 ....
0x00000030 02 00 00 00 ff ff ff ff ....
REG_BINARY 2 : (S)
0x00000000 70 00 72 00 65 00 70 00 72 00 6f 00 76 00 69 00 p.r.e.p.r.o.v.i.
0x00000010 73 00 69 00 6f 00 6e 00 65 00 72 00 2e 00 70 00 s.i.o.n.e.r...p.
0x00000020 73 00 31 00 00 00 80 00 32 00 00 00 00 00 00 00 s.1.....2.......
0x00000030 00 00 00 00 70 72 65 70 72 6f 76 69 73 69 6f 6e ....preprovision
0x00000040 65 72 2e 70 73 31 2e 6c 6e 6b 00 00 5a 00 08 00 er.ps1.lnk..Z...
0x00000050 04 00 ef be 00 00 00 00 00 00 00 00 2a 00 00 00 ....*...
0x00000060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
0x00000070 00 00 00 00 00 00 70 00 72 00 65 00 70 00 72 00 ......p.r.e.p.r.
0x00000080 6f 00 76 00 69 00 73 00 69 00 6f 00 6e 00 65 00 o.v.i.s.i.o.n.e.
0x00000090 72 00 2e 00 70 00 73 00 31 00 2e 00 6c 00 6e 00 r...p.s.1...l.n.
0x000000a0 6b 00 00 00 26 00 00 00 k...&...
REG_BINARY 0 : (S)
0x00000000 4f 00 50 00 45 00 4e 00 53 00 53 00 48 00 2e 00 O.P.E.N.S.S.H...
0x00000010 50 00 53 00 31 00 00 00 6a 00 32 00 00 00 00 00 P.S.1...j.2.....
0x00000020 00 00 00 00 00 00 4f 50 45 4e 53 53 48 2e 50 53 ......OPENSSH.PS
0x00000030 31 2e 6c 6e 6b 00 4c 00 08 00 04 00 ef be 00 00 1.lnk.L.........
0x00000040 00 00 00 00 00 00 2a 00 00 00 00 00 00 00 00 00 ......*.........
0x00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
0x00000060 4f 00 50 00 45 00 4e 00 53 00 53 00 48 00 2e 00 O.P.E.N.S.S.H...
0x00000070 50 00 53 00 31 00 2e 00 6c 00 6e 00 6b 00 00 00 P.S.1...l.n.k...
0x00000080 1e 00 00 00 ....
REG_BINARY 6 : (S)
0x00000000 70 00 6f 00 73 00 74 00 2d 00 77 00 69 00 6e 00 p.o.s.t.-.w.i.n.
0x00000010 2d 00 75 00 70 00 64 00 61 00 74 00 65 00 73 00 -.u.p.d.a.t.e.s.


kali-linux-2025.2-vmware...
1 2 3 4
kali@kali: ~/volatility2
File Actions Edit View Help
0x00000080 1e 00 00 00 ....
REG_BINARY 6 : (S)
0x00000000 70 00 6f 00 73 00 74 00 2d 00 77 00 69 00 6e 00 p.o.s.t.-.w.i.n.
0x00000010 2d 00 75 00 70 00 64 00 61 00 74 00 65 00 73 00 -.u.p.d.a.t.e.s.
0x00000020 2e 00 70 00 73 00 31 00 00 00 86 00 32 00 00 00 ..p.s.1.....2...
0x00000030 00 00 00 00 00 00 00 00 70 6f 73 74 2d 77 69 6e ....post-win
0x00000040 2d 75 70 64 61 74 65 73 2e 70 73 31 2e 6c 6e 6b -updates.ps1.lnk
0x00000050 00 00 5e 00 08 00 04 00 ef be 00 00 00 00 00 00 ..^.............
0x00000060 00 00 2a 00 00 00 00 00 00 00 00 00 00 00 00 00 ..*.............
0x00000070 00 00 00 00 00 00 00 00 00 00 00 00 70 00 6f 00 ....p.o.
0x00000080 73 00 74 00 2d 00 77 00 69 00 6e 00 2d 00 75 00 s.t.-.w.i.n.-.u.
0x00000090 70 00 64 00 61 00 74 00 65 00 73 00 2e 00 70 00 p.d.a.t.e.s...p.
0x000000a0 73 00 31 00 2e 00 6c 00 6e 00 6b 00 00 00 28 00 s.1...l.n.k...(.
0x000000b0 00 00 ..
REG_BINARY 1 : (S)
0x00000000 46 00 6c 00 6f 00 70 00 70 00 79 00 20 00 44 00 F.l.o.p.p.y...D.
0x00000010 69 00 73 00 6b 00 20 00 44 00 72 00 69 00 76 00 i.s.k...D.r.i.v.
0x00000020 65 00 20 00 28 00 41 00 3a 00 29 00 00 00 88 00 e...(.A.:.).....
0x00000030 32 00 00 00 00 00 00 00 00 00 00 00 46 6c 6f 70 2...........Flop
0x00000040 70 79 20 44 69 73 6b 20 44 72 69 76 65 20 28 41 py.Disk.Drive.(A
0x00000050 29 2e 6c 6e 6b 00 60 00 08 00 04 00 ef be 00 00).lnk.`.........
0x00000060 00 00 00 00 00 00 2a 00 00 00 00 00 00 00 00 00 ......*.........
0x00000070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
0x00000080 46 00 6c 00 6f 00 70 00 70 00 79 00 20 00 44 00 F.l.o.p.p.y...D.
0x00000090 69 00 73 00 6b 00 20 00 44 00 72 00 69 00 76 00 i.s.k...D.r.i.v.
0x000000a0 65 00 20 00 28 00 41 00 29 00 2e 00 6c 00 6e 00 e...(.A.)...l.n.
0x000000b0 6b 00 00 00 28 00 00 00 k...(...
REG_BINARY 7 : (S)
0x00000000 57 00 69 00 6e 00 64 00 6f 00 77 00 73 00 20 00 W.i.n.d.o.w.s...
0x00000010 55 00 70 00 64 00 61 00 74 00 65 00 00 00 74 00 U.p.d.a.t.e...t.
0x00000020 32 00 00 00 00 00 00 00 00 00 00 00 00 57 69 6e 2............Wind
0x00000030 6f 77 73 20 55 70 64 61 74 65 2e 6c 6e 6b 00 00 ows.Update.lnk..
0x00000040 52 00 08 00 04 00 ef be 00 00 00 00 00 00 00 00 R...............
0x00000050 2a 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 *...............
0x00000060 00 00 00 00 00 00 00 00 00 00 00 57 00 69 00 6e ....W.i.n
0x00000070 64 00 6f 00 77 00 73 00 20 00 55 00 70 00 64 00 d.o.w.s...U.p.d.
0x00000080 61 00 74 00 65 00 2e 00 6c 00 6e 00 6b 00 00 00 a.t.e...l.n.k...
0x00000090 22 00 00 00 "...
REG_BINARY 8 : (S)
0x00000000 44 00 69 00 73 00 70 00 6c 00 61 00 79 00 00 00 D.i.s.p.l.a.y...
0x00000010 5e 00 32 00 00 00 00 00 00 00 00 00 00 00 44 69 ^.2...........Di
0x00000020 73 70 6c 61 79 2e 6c 6e 6b 00 44 00 08 00 04 00 splay.lnk.D.....
0x00000030 ef be 00 00 00 00 00 00 00 00 2a 00 00 00 00 00 ....*.....



Output identifies the registry hive as:

- Hive: C:\Users\IEUser\ntuser.dat
- Registry key: Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs
- Last updated: 2019-01-06 14:32:14 UTC
- Registry value: MRUListEx, which records the ordering of entries in the RecentDocs list.
- Subkeys: .PS1, .txt, and Folder.

The binary registry values can be decoded to identify several recorded items:

Entry	Identified item

0	OPENSSH.PS1
2	preprovisioner.ps1
6	post-win-updates.ps1
10	Readme.txt
4	eula.txt
1	Floppy Disk Drive (A:)
7	Windows Update
8	Display
9	Network and Internet
3	System and Security
5	System32
11	32Bit
12	All Control Panel Items

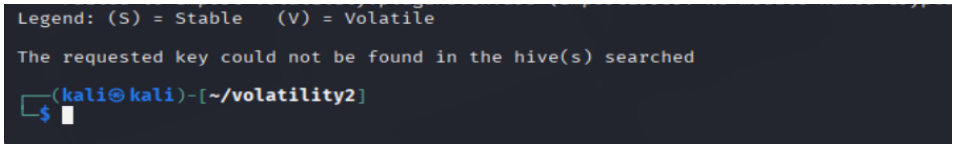
These names are decoded from the binary data shown in your output. Many are Windows shortcuts or system-related entries rather than evidence of a specific document being opened. The presence of PowerShell script shortcuts is worth recording, but does not establish that the scripts were executed.

The key's last-updated timestamp is earlier than the USB-related registry updates around 15:02–15:03 UTC. It should not be treated as the time any particular file was opened.

Examination of the RecentDocs .docx Registry Subkey

The printkey plugin in Volatility 2.6.1 was used to examine Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs\.docx within the NTUSER.DAT registry hive associated with the IEUser profile. Volatility returned the

message, “The requested key could not be found in the hive(s) searched.” Accordingly, no entries could be recovered from this specific .docx RecentDocs subkey in the examined memory image.



This result is limited to the requested registry path and does not demonstrate that Microsoft Word documents were absent from the system, that secret_file.docx did not exist, or that the reported copy operation did not occur. The parent RecentDocs key contained other entries, but secret_file.docx was not identified in the displayed values. The result was therefore recorded as an absence of recoverable evidence from this particular registry location, rather than as evidence disproving the suspected file transfer.

RunMRU Registry Examination

The registry key Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU was examined in the NTUSER.DAT hive associated with the IEUser profile using the Volatility 2.6.1 printkey plugin. Volatility successfully identified the key and reported its last-updated timestamp as 23 September 2015 at 10:16:38 UTC. However, no subkeys or values were recovered from the examined key. Consequently, this artefact provided no usable Run-dialog command history for the investigation. The timestamp predates the January 2019 activity reflected in other recovered artefacts and cannot be associated with the suspected transfer. The absence of values does not establish that no commands were executed on the system, because commands may have been entered through other interfaces or their records may not have been preserved in the available memory image. This finding was therefore documented as a limitation of the available RunMRU evidence and does not negate the command prompt history showing a copy command directed to drive F:.



- Registry hive: C:\Users\IEUser\ntuser.dat
- Registry key: Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU
- Last updated: 23 September 2015, 10:16:38 UTC
- Recovered subkeys: None
- Recovered values: None

TypedPaths Registry Examination

The registry key Software\Microsoft\Windows\CurrentVersion\Explorer\TypedPaths was examined in the NTUSER.DAT hive associated with the IEUser profile using the Volatility 2.6.1 printkey plugin. The key was successfully identified and reported as last updated on 21 September 2015 at 09:45:19 UTC. No subkeys or values were recovered. Consequently, the examination did not yield any manually entered Explorer paths that could be associated with the suspected file transfer. The timestamp substantially predates the January 2019 artefacts examined elsewhere in this investigation and was not treated as evidence of activity during the relevant period. The absence of values does not establish that the user did not navigate to files or folders, because this registry key captures only a limited category of Explorer activity and may not retain a complete history. This finding was recorded as a limitation of the available user-navigation evidence and does not contradict the independently recovered command prompt history and USB-device registry artefacts.

```
Legend: (S) = Stable   (V) = Volatile

----------------------------------
Registry: \??\C:\Users\IEUser\ntuser.dat
Key name: TypedPaths (S)
Last updated: 2015-09-21 09:45:19 UTC+0000

Subkeys:

Values:
```

Windows User Profile Registry Examination

The Microsoft\Windows NT\CurrentVersion\ProfileList registry key was examined in the system SOFTWARE hive using the Volatility 2.6.1 printkey plugin. The key was reported as last updated on 21 September 2015 at 09:50:52 UTC. Five SID subkeys were recovered: S-1-5-18, S-1-5-19, S-1-5-20, S-1-5-21-1716914095-909560446-1177810406-1000, and S-1-5-21-1716914095-909560446-1177810406-1002. The first three correspond to standard Windows system and service accounts, while the remaining two represent user-specific account identifiers. The key's values also recorded the configured profile locations, including %SystemDrive%\Users for the profiles directory. The parent key did not directly establish the usernames associated with the two user-specific SIDs; further examination of their individual subkeys is required to identify their recorded profile paths. These findings establish the presence of profile configuration entries but do not independently identify the person operating the system during the suspected transfer.

```
Legend: (S) = Stable   (V) = Volatile

----------------------------------
Registry: \SystemRoot\System32\Config\SOFTWARE
Key name: ProfileList (S)
Last updated: 2015-09-21 09:50:52 UTC+0000

Subkeys:
  (S) S-1-5-18
  (S) S-1-5-19
  (S) S-1-5-20
  (S) S-1-5-21-1716914095-909560446-1177810406-1000
  (S) S-1-5-21-1716914095-909560446-1177810406-1002

Values:
REG_EXPAND_SZ ProfilesDirectory : (S) %SystemDrive%\Users
REG_EXPAND_SZ Default           : (S) %SystemDrive%\Users\Default
REG_EXPAND_SZ Public            : (S) %SystemDrive%\Users\Public
REG_EXPAND_SZ ProgramData       : (S) %SystemDrive%\ProgramData
```

Output shows the registry key:

- Hive: SOFTWARE
- Registry path: Microsoft\Windows NT\CurrentVersion\ProfileList
- Last updated: 21 September 2015, 09:50:52 UTC

The key contains these SIDs:

SID	Initial interpretation
S-1-5-18	Local System account
S-1-5-19	Local Service account
S-1-5-20	Network Service account
S-1-5-21-1716914095-909560446-1177810406-1000	User-specific account SID
S-1-5-21-1716914095-909560446-1177810406-1002	User-specific account SID

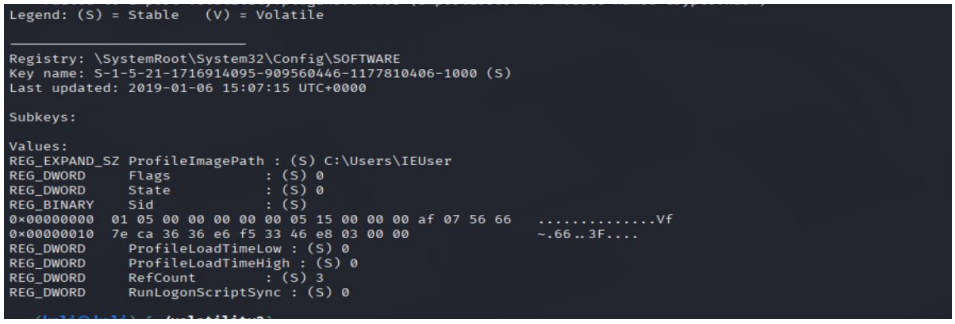
The ProfileList key also shows the standard profile directory configuration, including %SystemDrive%\Users as the profiles directory.

Important: The SIDs ending in 1000 and 1002 are account identifiers, but the parent key alone does not tell us the corresponding usernames. We need to inspect each SID subkey and its ProfileImagePath value to establish which profile directory it represents.

Mapping of the User SID to the IEUser Profile

The ProfileList subkey S-1-5-21-1716914095-909560446-1177810406-1000 was examined in the system SOFTWARE hive using the Volatility 2.6.1 printkey plugin. Its ProfileImagePath value was recovered as C:\Users\IEUser, establishing that the SID maps to the IEUser profile directory. The key was reported as last updated on 6 January 2019 at 15:07:15 UTC. Additional values included Flags set to 0, State set to 0, and RefCount set to 3; these were recorded as profile-management metadata and were not interpreted as evidence of user identity or incident activity. The registry timestamp coincides with the previously observed start time of the FTK Imager.exe process, creating a temporal correlation between the two artefacts. However, the available evidence does not establish that the process launch caused the registry update or that either event was

directly related to the file-copy operation. The SID-to-profile mapping corroborates the use of the IEUser profile in the examined Windows environment, but it does not identify the real-world person who operated the computer.



- SID: S-1-5-21-1716914095-909560446-1177810406-1000
- ProfileImagePath: C:\Users\IEUser
- Last updated: 2019-01-06 15:07:15 UTC
- State: 0
- RefCount: 3

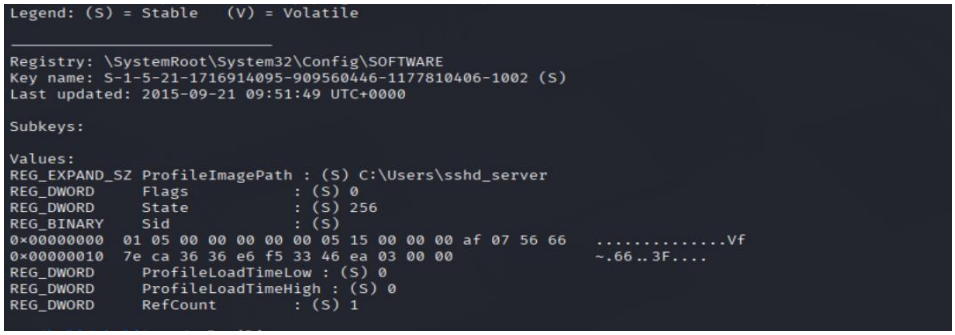
The ProfileImagePath is the strongest result here: it links this SID to the IEUser profile. The Sid binary value is consistent with the SID represented by the registry subkey. The RefCount value is a registry profile-management value and should not be interpreted as a count of users or people involved in the incident.

The timestamp matches the recorded start time of FTK Imager.exe in the earlier process analysis. Treat this as a correlation between two artefacts, not proof of a causal relationship.

Mapping of the Second User SID to the sshd_server Profile

The ProfileList subkey S-1-5-21-1716914095-909560446-1177810406-1002 was examined in the system SOFTWARE hive using the Volatility 2.6.1 printkey plugin. Its ProfileImagePath value was recovered as C:\Users\sshd_server, mapping the SID to the sshd_server profile directory. The key was reported as last updated on 21 September 2015 at 09:51:49 UTC and contained Flags set to 0, State set to 256, and RefCount set to 1. These values were recorded as registry profile metadata and were not treated as

proof of a particular logon or user action. The presence of this profile is consistent with the separate sshd_server user hive and the previously observed OpenSSH-related processes and TCP port 22 listener. However, these artefacts establish only that an SSH service and associated profile were present in the examined environment; they do not establish a successful remote login, an SSH-based file transfer, or attribution of the suspected copy operation to this account. Further session or authentication evidence would be required to support those conclusions.



- SID: S-1-5-21-1716914095-909560446-1177810406-1002
- ProfileImagePath: C:\Users\sshd_server
- Last updated: 2015-09-21 09:51:49 UTC
- State: 256
- RefCount: 1

The profile path links this SID to the sshd_server account profile. The earlier process analysis recovered sshd.exe (PID 2016), and the network scan showed it listening on TCP port 22. Together, these artefacts establish that OpenSSH-related components and an sshd_server profile were present in the examined environment. They do not establish that a remote party logged in, that an SSH session transferred the document, or that the account was responsible for copying it. The State value of 256 is profile metadata.

ShellBags Plugin Availability and Limitation

A check was performed to determine whether the ShellBags plugin was available in the Volatility 2.6.1 environment. The command python2 vol.py --info was used to list the available plugins, with the output filtered for entries containing “shellbags”. The check

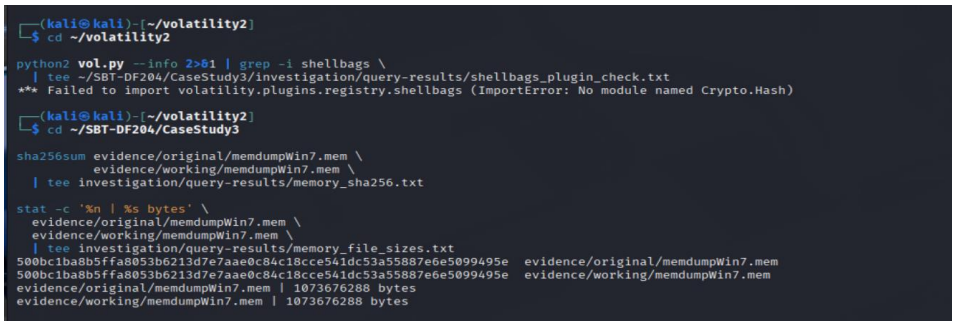
returned the following error: Failed to import volatility.plugins.registry.shellbags (ImportError: No module named Crypto.Hash).

This error indicates that the ShellBags plugin could not be loaded because the required Crypto.Hash module was unavailable in the Python 2 environment. Consequently, ShellBags analysis could not be performed during this phase of the investigation. No conclusion should therefore be drawn about the presence or absence of relevant ShellBags artefacts. The check and its output were saved to investigation/query-results/shellbags_plugin_check.txt for documentation and reproducibility. This limitation will be disclosed in the final report

Commented [KO1]:

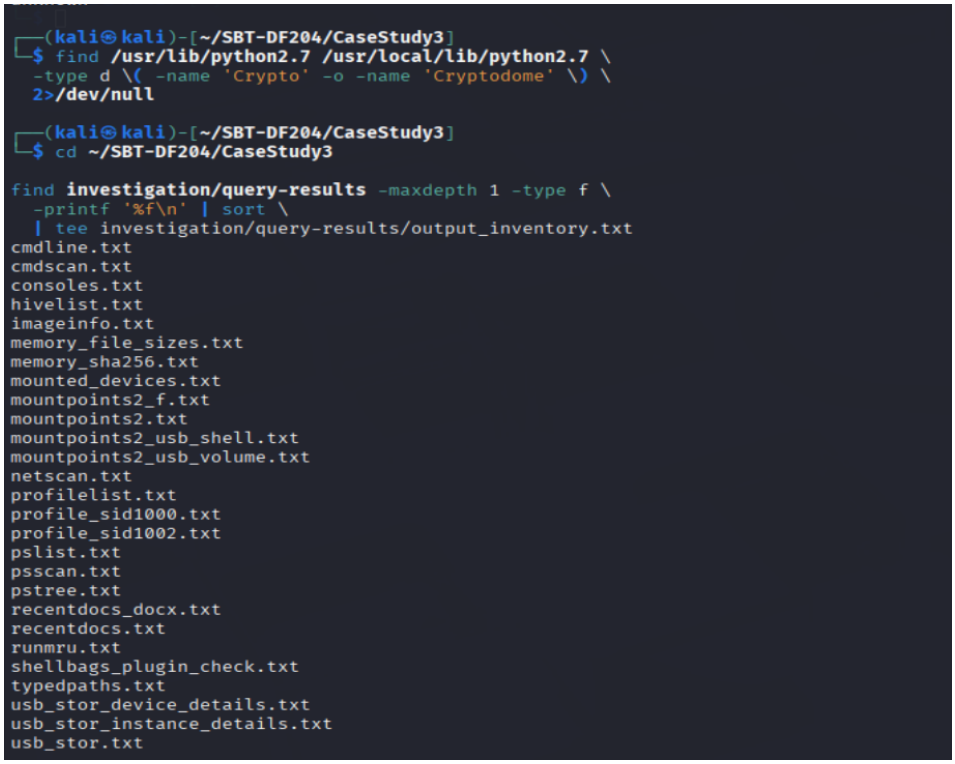
Evidence Integrity Verification

The original memory image and its working copy were subjected to SHA-256 hash verification to assess whether the working copy had changed during preparation for analysis. The original evidence file was located at evidence/original/memdumpWin7.mem, while the working copy was located at evidence/working/memdumpWin7.mem.



The SHA-256 hash calculated for both files was 500bc1ba8b5ffa8053b6213d7e7aae0c84c18cce541dc53a55887e6e5099495e. Both files also had an identical size of 1,073,676,288 bytes. The matching hashes and file sizes establish that the two files were byte-for-byte identical at the time of verification. This supports the integrity of the working copy relative to the original evidence image. The results were recorded in investigation/query-results/memory_sha256.txt and investigation/query-results/memory_file_sizes.txt.

This comparison confirms consistency between the two available files at the time of hashing; it does not independently establish the integrity of the evidence before it was received or prove the complete chain of custody.



Chronological Timeline of Relevant Artefacts

The following timeline was reconstructed from the timestamps recovered during memory and registry analysis. All timestamps are presented in Coordinated Universal Time (UTC). The timeline combines registry key update times with process creation times and a network artefact timestamp. These timestamps represent the recorded times associated with the respective artefacts and should not automatically be interpreted as exact times of user actions.

At 15:02:54 on 6 January 2019, the USBSTOR and MountedDevices registry keys were last updated. The USBSTOR entries identified a device described as General UDisk USB

Device, while the MountedDevices data associated the F: drive mapping with the same USB storage device instance. At 15:03:06, the MountPoints2\F registry key was last updated. At 15:03:07, a volume GUID entry associated with the USB device was last updated under the user's MountPoints2 registry data. Collectively, these artefacts support the conclusion that Windows maintained registry records associating the removable storage device with the system's drive and volume mappings. However, registry key timestamps alone do not establish the exact time the device was physically connected.

At 15:06:11, the cmd.exe process was recorded as having started. The recovered console history associated with this command interpreter included the commands ipconfig, cd Downloads, and copy secret_file.docx F:. The console screen buffer also contained the message 1 file(s) copied. This is the strongest direct evidence recovered in the investigation that a copy operation targeting drive F: was recorded in the captured console state. The recovered history does not provide a reliable individual timestamp for the copy command, so the command cannot be assigned an exact execution time from the available evidence.

At 15:07:15, the FTK Imager.exe process was recorded as having started. The IEUser profile registry key ending in SID 1000 was also last updated at this time. This temporal correlation is documented for completeness, but it does not establish that FTK Imager caused the registry update or that the application was used to perform the recorded file copy.

At 15:08:32, a UDP network artefact associated with Chrome process PID 2388 was recorded. This timestamp provides additional context about system activity after the preceding events, but the available UDP artefact does not demonstrate that secret_file.docx was transmitted over the network.

Overall, the reconstructed timeline shows a sequence of USB-related registry updates followed by the recorded creation of a command interpreter and other system activity. When considered alongside the recovered console history, the USB registry mappings provide corroborating context for a file copy to the removable storage device. The available timestamps do not establish an exact time for the copy operation, and the

investigation did not recover conclusive evidence that the file was subsequently transmitted over a network.

CONCLUSION

The memory-forensics investigation recovered evidence consistent with a file-copy operation from the IEUser profile's Downloads directory to a drive mapped as F:. The most direct artefact was the console history associated with cmd.exe (PID 3996), which contained the command copy secret_file.docx F: and the message 1 file(s) copied. The USBSTOR and MountedDevices registry records identified a General UDisk USB Device and associated its device instance with the F: drive mapping. MountPoints2 records provided further corroborating context for the relationship between the user profile and the removable volume.

Process and network analysis identified sshd.exe (PID 2016) listening on TCP port 22 and recovered closed TCP records involving the local address 192.168.56.8 and remote address 192.168.56.5 on port 80. However, these network artefacts do not demonstrate that the document was transferred over a network. The available records also do not establish the contents of the document, its classification, the identity of the person who entered the command, or whether the activity violated an applicable authorisation policy.

Confidence assessment: There is high confidence that the captured console state contains a file-copy command targeting F: and a corresponding console message reporting that one file was copied. There is moderate-to-high confidence that the command and registry artefacts are consistent with an attempted or reported copy to the identified USB storage device. There is insufficient evidence to conclude definitively that an unauthorised transfer of confidential information occurred or to attribute the activity to a specific individual.

The investigation was limited by the absence of a reliable per-command timestamp, the inability to complete ShellBags analysis because of a missing Python dependency, and the distinction between a reported console result and independent verification of the destination file. The operating-system profile identified in the image also differed from the Windows 10 description in the scenario. These limitations have been retained in the

interpretation to ensure that the conclusion remains proportionate to the available evidence.